

KONU 1: Cargo ve Proje Yapısı

Kategori: Araçlar | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```
# Terminalde (proje klasörünün DIŞINDA) çalıştır:
cargo new hello_cargo      # yeni proje oluşturur
cd hello_cargo             # Cargo.toml'un olduğu klasöre gir
cargo run                  # derle + çalıştır
```

```
// src/main.rs — cargo new bunu otomatik oluşturur
fn main() {
    println!("Hello, Cargo!"); // ekrana yazar
}
// Çıktı: Hello, Cargo!
```

cargo new komutu bir package (paket) oluşturur. İçinde manifest (bildirim dosyası) olan Cargo.toml ve kaynak kod olan src/main.rs bulunur. cargo run önce compiler (derleyici) ile build (derleme) yapar, sonra oluşan binary'yi (çalıştırılabilir dosya) çalıştırır. Çıktılar target/debug/ altına yazılır ve bu klasörü elle düzenlemezsin. cargo check binary üretmeden sadece hataları kontrol eder, bu yüzden cargo build'den hızlıdır.

1.96.0 notu: x86_64 Linux'ta LLD artık varsayılan linker (bağlayıcı); özellikle büyük binary'lerde ve incremental rebuild'lerde linking daha hızlı. Komutların kendisi değişmedi.

KATMAN 2 — RECALL KARTLARI

- cargo new hello_cargo → yeni klasör + Cargo.toml + src/main.rs + .gitignore + git repo

İlgili: cargo init

- cargo init → mevcut klasörü Cargo projesi yapar

Hata: error: cargo init cannot be run on existing Cargo packages

- cargo check → binary üretmez, sadece hata kontrolü (en hızlısı)

İlgili: cargo build

- cargo build → target/debug/hello_cargo üretir; ilk build'de Cargo.lock oluşur

- cargo run → build (gerekirse) + çalıştır

İlgili: cargo run --bin isim

- cargo build --release → target/release/ altına optimize binary

İlgili: profile (profil), opt-level

- Cargo.toml iskeleti:

```
[package]
name = "hello_cargo"
version = "0.1.0"
edition = "2024"    # edition (sürüm dönemi)

[dependencies]
```

- cargo add rand → [dependencies] altına dependency (bağımlılık) ekler

İlgili: cargo remove rand

- cargo clean → target/ klasörünü siler

- Hata: could not find Cargo.toml in current directory or any parent directory → yanlış klasördesin

- Hata: E0601 → crate (kutu) içinde fn main yok

GoT bağlantı haritası:

```
cargo new Cargo.toml [package] edition
      [dependencies] cargo add
      src/main.rs cargo check cargo build cargo run
                                --release
```



KATMAN 3 — DERİN TEKNİK NOT

(Bu konuda stack/heap yok, bu yüzden "bellek" adımı diske ve process'e uyarlandı.)

Adım 1 — Disk ve Process Seviyesi

- Önce soru: "Diskte ne oluyor?" Cevap: target/debug/ altında binary, deps/ (ara dosyalar), incremental/ (artımlı derleme cache'i) ve .fingerprint/.
- Cargo.lock kaynak koduyla birlikte tutulur ve tüm dependency'lerin tam sürümlerini kilitler. Böylece reproducible build (tekrarlanabilir derleme) mümkün olur.
- Binary çalışınca OS bir process açar ve stack'i kurar. fn main'i doğrudan OS değil, Rust'ın std runtime başlangıç kodu (lang_start) çağırır.
- Debug ve release farkı:

debug (cargo build)	release (--release)
opt-level	0
debug-assertions	açık
overflow-checks	açık (integer overflow (tam sayı taşması) → panic)
Derleme süresi	kısa
Çalışma hızı	yavaş

Adım 2 — Compiler Davranışı

- Önce soru: "Kim derliyor?" Cevap: Cargo derlemez, her crate için rustc çağırır.
- cargo build -v ile çağrılan rustc komutunu görebilirsin (--crate-name, --edition=2024, --crate-type bin gibi bayraklar).
- cargo check, rustc'ı sadece metadata üretecek şekilde çalıştırır; code generation ve linking atlanır. Hızı buradan gelir.
- Dosya değişmediyse Cargo yeniden derlemez (fingerprint kontrolü).
- MIR görmek için: cargo rustc -- --emit=mir (çıktı target/debug/deps/ altına yazılır).

Adım 3 — Hata Kodları

- E0601: binary crate'te main fonksiyonu yok. fn main() {} ekle veya dosya adını/konumunu kontrol et. Detay için rustc --explain E0601.
- Cargo hatası (E kodu yok): no targets specified in the manifest → src/main.rs veya src/lib.rs eksik.
- Cargo hatası: feature edition2024 is required → Cargo eski. rustup update ile çöz.

Adım 4 — "Neden Böyle?"

- Çözülen problem: build, dependency yönetimi ve proje düzeni için tek standart araç.
- C/C++'ta Make/CMake yazarsın, kütüphaneleri elle bulur ve bağlarsın. Rust'ta Cargo.toml + cargo build yeter.
- Trade-off: Rust'ta derleme yavaş, çalışma hızlı. Bu yüzden iki profil var: geliştirirken debug, yayınlarken release.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni Range* tipleri, assert_matches! makrosu, Wasm linker güncellemeleri) temel Cargo proje yapısını değiştirmiyor. Wasm hedeflerinde linker davranışı güncellendi.
- Yeni Range* tipleri ve assert_matches! makrosu cargo publish --workspace kullanımını değiştirmiyor.
- x86_64-apple-darwin, Tier 1'den Tier 2'ye indirildi; derlenmiş sürümler dağıtılmaya devam ediyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: check / build / run farkı

- Neden unutuluyor: üçü de "derliyor" gibi duruyor.
- Yol 1 (analoji): check = imla kontrolü, build = kitabı bas, run = bas + oku.
- Yol 2 (kod): cargo check sonrası target/debug/hello_cargo oluşmaz.
- Yol 3 (hata): ./target/debug/hello_cargo → No such file or directory (sadece check yaptıysan)

Sık Unutulan 2: Komut nerede çalışıyor?

- Neden unutuluyor: cargo new projenin dışında, cargo run içinde çalışır.
- Yol 1 (görsel): Cargo.toml'u gören klasör = çalışma klasörü.
- Yol 2 (kod): cd hello_cargo && cargo run
- Yol 3 (hata): could not find Cargo.toml in current directory or any parent directory

Sık Unutulan 3: Release binary nerede?

- Neden unutuluyor: cargo run --release çalıştırır, ama yolunu söylemez.
- Yol 1 (analoji): debug = taslak klasörü, release = baskı klasörü.
- Yol 2 (kod): ./target/release/hello_cargo
- Yol 3 (hata): --release olmadan target/release/ boştur.

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da Cargo ve proje yapısı konusunu en detaylı şekilde anlat. Şunları kapsa:

1. DİSK/PROCESS SEVİYESİ: target/ klasöründe ne var, Cargo.lock ne işe yarar, binary çalışınca process'te ne oluyor?
2. COMPILER DAVRANIŞI: Cargo, rustc'ı nasıl çağırıyor? cargo build -v çıktısını satır satır açıkla. check, build, release farkı nedir? Incremental compilation nasıl çalışıyor?
3. HATA KODLARI: E0601 ve sık görülen Cargo hataları. Her biri için: ne zaman, neden, nasıl çözülür. `rustc --explain E0601` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: Make/CMake ile aynı iş nasıl yapılır? Cargo hangi problemi çözüyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: mutfak, fabrika hattı).
6. RUST 1.96.0: LLD varsayılan linker değişikliği ve cargo publish --workspace gibi yeniliklerin günlük kullanıma etkisi.
7. PRATİK: 3 tane örnek ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Komutları ve kodu çalıştırılabilir yaz.



GÖREV (Boş terminal + boş dosya)



Kolay

1. cargo new rust_notes aç, içine gir, cargo run ile çalıştır.
2. Cargo.toml içindeki name, version, edition satırlarını bul.



Orta

3. main.rs'e 2 tane println! ekle. Sırayla cargo check, cargo build, cargo run çalıştır.
4. cargo add rand ile dependency ekle, Cargo.toml'da gör, sonra cargo remove rand ile sil.



Zor

5. `src/bin/second.rs` oluştur (içinde `fn main`), `cargo run --bin second` ile çalıştır. Sonra `cargo build --release` yap ve `debug/release` binary boyutlarını `ls -lh target/debug target/release` ile karşılaştır.

Başarı kriteri: Hepsini hatasız yaparsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. Proje oluştur ve çalıştır.
2. `Cargo.toml` iskeletini ezberden yaz.
3. `check` / `build` / `run` / `--release` farkını söyle ve binary yollarını yaz.

KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Hello, Cargo!
- Cargo Book — Package Layout
- Cargo Book — Creating a New Package
- Cargo Book — Profiles

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes
- Rust Blog — LLD on 1.96.0 stable

Hata Kodları

- E0601

Ek Kaynaklar

- Rust By Example — Cargo

Not: Sürüm notu kaynakları bu oturumda aramayla doğrulandı. Rust Book, Cargo Book ve E0601 bağlantıları standart `doc.rust-lang.org` adresleridir. Kodları ve komutları kendi makinende bir kez çalıştırarak kontrol et.

KONU 2: println!() ve Formatlama

Kategori: Temel | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```
fn main() {
    let name = "Ferris";
    let age = 7;
    let pi = 3.14159;

    println!("Hello, {}!", name);           // positional: sırayla doldurur
    println!("{name} is {age} years old"); // inline: değişken adını direkt yaz
    println!("pi = {pi:.2}");               // virgülden sonra 2 basamak
    println!("{:>6}", age);                 // sağa hizala
    println!("{:<6}", age);                 // sola hizala
    println!("{:^6}", age);                 // ortala
    println!("{:06}", age);                 // sıfırla doldur
    println!("{:?}", name);                 // Debug formatı
    println!("{}", "{} are braces");       // süslü parantezi yazdırma
}

// Çıktı:
// Hello, Ferris!
// Ferris is 7 years old
// pi = 3.14
// [      7]
// [7      ]
// [ 7     ]
// [000007]
// "Ferris"
// {} are braces
```

`println!` bir macro (makro) olduğu için sonunda `!` var ve ekrana yazıp satır sonu ekler. Metnin içindeki `{}` işaretleri placeholder (yer tutucu) görevi görür ve argümanlarla sırayla doldurulur. `{name}` yazarsan aynı isimli değişken otomatik alınır. `:` işaretinden sonra width (genişlik), alignment (hizalama) ve precision (hassasiyet) gibi ayarlar yazılır. `{:?}` Debug biçimidir ve string'i tırnaklı gösterir.

1.96.0 notu: Formatlama kuralları aynı; 1.96.0'ın öne çıkan değişiklikleri (LLD, workspace publish) bu konuyu etkilemiyor.

KATMAN 2 — RECALL KARTLARI

`println!("{}", x)` → Display biçimi, sonuna `\n` ekler

İlgili: `print!`

`print!("no newline")` → satır sonu eklemez

Hata: çıktı geç görünür → stdout tamponlanır, flush gerekir (aşağıya bak)

`eprintln!("error")` → stderr'e yazar

İlgili: `eprint!`

`let s = format!("{}", x);` → ekrana yazmaz, String döndürür

İlgili: heap allocation

`{0} {1} {0}` → numaralı argüman; `{n} {m}` → isimli argüman

```
println!("{}", "a", "b"); // a b a
println!("{}", x, y = 2); // 1 2
```

- Hizalama: `{:<8}` sol, `{:>8}` sağ, `{:^8}` orta, `{:*^9}` yıldızla doldurarak orta
- Sayı ayarları: `{:+}` işaret, `{:05}` sıfır dolgu, `{:.3}` 3 basamak, `{:8.2}` genişlik 8 + 2 basamak
- Taban ve bilimsel: `{:b}` binary, `{:x}` hex, `{:#x}` 0x önekli hex, `{:o}` octal, `{:e}` bilimsel
- Dinamik genişlik: `println!("{}", x, w = 8);` veya `println!("{}", x, w, "p$");`
- Debug: `{:?}` tek satır, `{:#?}` pretty-print (okunaklı yazdırma)

```
#[derive(Debug)]
struct Point { x: i32, y: i32 }
// {:?} → Point { x: 1, y: 2 }
```

Hata: E0277: Point doesn't implement std::fmt::Display ({} kullandın) veya Debug (derive unuttun)

- dbg!(x) → stderr'e dosya:satır ile yazar ve değeri geri döndürür

- flush için: use std::io::Write; std::io::stdout().flush().unwrap();

Hata: E0599: no method named flush found → use std::io::Write; eksik

- Hata: {foo} yazdın ama foo yok → E0425: cannot find value foo in this scope

GoT bağlantı haritası:

```
println!  {}  Display
           {:?}  Debug  #[derive(Debug)]
           {:#?}  pretty-print
           {name}  inline capture
           {:>8.2}  width / precision
print!  flush gerekebilir
eprintln!  stderr
format!  String (heap)
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Bellekte ne oluyor?" Cevap: println! argümanları taşımaz, referans alır.

```
let s = String::from("hi");
println!("{s}");
println!("{s}"); // OK: s hâlâ geçerli (move olmadı)
```

- format_args! stack'te küçük bir fmt::Arguments yapısı kurar. İçinde sabit metin parçalarına referanslar ve her argüman için (pointer + formatlayıcı fonksiyon pointer'ı) çifti vardır. Bu aşamada heap kullanılmaz.
- Çıktı stdout'a gider. Stdout'un arkasında küçük bir buffer (tampon) vardır ve \n görünce flush edilir. print! satır sonu içermediği için çıktı hemen görünmeyebilir.
- format! ise sonucu heap'te büyüyen bir String'e yazar. Stack'te (ptr, len, cap), heap'te metin tutulur.

Adım 2 — Compiler Davranışı

- Önce soru: "Compiler ne kontrol ediyor?" Cevap: format string compile time'da parse edilir.

- Placeholder sayısı ile argüman sayısı eşleşmezse hata verir.
- Kullanılmayan argüman hata olur.
- Her placeholder için ilgili trait kontrol edilir (`{}` → `Display`, `{:?}` → `Debug`, `{:x}` → `LowerHex`).
- Format string string literal olmalıdır; `println!(s)` derlenmez.
- Inline capture sadece identifier alır; `{x.y}` ya da `{a + b}` yazamazsın. Önce değişkene ata.
- Macro'nun açılmış hâlini görmek için nightly gerekir: `cargo +nightly rustc --Zunpretty=expanded`. MIR için: `cargo rustc -- --emit=mir`.

Adım 3 — Hata Kodları ve Mesajlar

- E0277: `{}` ile yazdığının tipte `Display` yok. Çözüm: `{:?}` + `#[derive(Debug)]` kullan veya `impl fmt::Display` yaz. Detay: `rustc --explain E0277`.
- E0599: `flush` çağırıyorsun ama `Write` trait'i kapsamda değil. Çözüm: `use std::io::Write;`
- E0425: `{foo}` içindeki `foo` tanımlı değil. Çözüm: değişkeni tanımla veya `foo = ...` ile isimli argüman ver.
- Kodsuz hatalar:
- 2 positional arguments in format string, but there is 1 argument
- argument never used
- format argument must be a string literal
- invalid format string: expected `'` (süslü parantezi `{{}}` ile kaçırmadın)

Adım 4 — "Neden Böyle?"

- Çözülen problem: C'deki `printf("%d", x)` tip güvensizliği. `%d` yazıp `char*` verersen derleyici çoğu zaman susar ve davranış tanımsız olur (undefined behavior).
- Rust'ta `println!("{}", x)` tip kontrolünü compile time'da yapar; runtime'da format string yorumlanmaz.
- Trade-off: macro olduğu için format string çalışma zamanında değiştirilemez. Dinamik biçim gerekiyorsa `{:w$}` gibi parametreleri kullan.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni `Range*` tipleri, `assert_matches!` makrosu, Wasm linker güncellemeleri) formatlama macro'larını etkilemiyor.
- Format args capture, Rust 1.58 duyurusunda "captured identifiers in format strings" başlığıyla yer alır. Bu yüzden `{name}` güvenle kullanılabilir.
- Width ve precision parametreleri de capture edilebilir (`{x:width-precision}`); aynı isimli açık bir named argument varsa o önceliklidir.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: {} mi {:?} mi?

- Neden unutuluyor: ikisi de "yazdır" gibi görünüyor.
- Yol 1 (görsel): ? = soru işareti = "içini göster, debug için".
- Yol 2 (kod): `println!("{:?}", vec![1, 2]);` → [1, 2], ama `println!("{}", vec![1, 2]);` derlenmez.
- Yol 3 (hata): `E0277: Vec<i32> doesn't implement std::fmt::Display`

Sık Unutulan 2: Süslü parantezi yazdırmak

- Neden unutuluyor: { normalde placeholder açar.
- Yol 1 (analoji): iki kapı kilidi, {{ ve }} ikişer kez yazılır.
- Yol 2 (kod): `println!("{}", {{}});` → {}
- Yol 3 (hata): `println!("{}",);` → invalid format string: expected '}' but string was terminated

Sık Unutulan 3: Sözdizimi sırası

- Neden unutuluyor: fill align sign # 0 width .precision type sırası uzun.
- Yol 1 (görsel): önce nasıl dolsun (fill+align), sonra ne kadar geniş (width), sonra kaç basamak (.precision), en sonda hangi tür (x, b, e, ?).
- Yol 2 (kod): `{:^10.2}` → fill ^, orta, genişlik 10, 2 basamak.
- Yol 3 (hata): `{:.2>8}` yanlış sıra → invalid format string

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da `println!()` ve formatlama konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: `println!` ve `format!` çağrısında stack'te ne var, heap'te ne var?
`fmt::Arguments` yapısı nedir? Argümanlar neden move edilmez?
2. COMPILER DAVRANIŞI: Format string compile time'da nasıl parse ediliyor?
`Display`, `Debug`, `LowerHex` trait'leri nasıl seçiliyor? Macro açılımı nasıl görünür?
(`-Zunpretty=expanded`, `--emit=mir`)
3. HATA KODLARI: E0277, E0599, E0425 ve format string hata mesajları.
Her biri için: ne zaman, neden, nasıl çözülür.
``rustc --explain EXXX`` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: `printf` ve `std::cout` ile aynı işlem nasıl yapılır?
Rust neden farklı? Hangi problemi çözüyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: şablon mektup, doldurulacak form).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. `name` ve `age` değişkenlerini tanımla; `println!` ile inline capture kullanarak "Ali is 20 years old" yazdır.
2. `pi = 3.14159` değerini 2 basamakla yazdır.



Orta

3. `#[derive(Debug)] struct Point { x: i32, y: i32 }` tanımla; `{:?}` ve `{:#?}` ile yazdır.
4. 255 sayısını decimal, binary (`{:b}`), hex (`{:#x}`) ve octal olarak tek tek yazdır.



Zor

5. `{{ }}` kullanarak `{value}` metnini harfi harfine yazdır. Sonra `w = 10` değişkeniyle dinamik genişlikte (`{:>w$}`) sağa hizalı bir sayı yazdır ve `print! + flush` ile aynı satırda "Loading..." yazıp devam et.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. `{}` ve `{:?}` farkını gösteren iki satır yaz.
2. `{:*^9.2}` ile bir `f64` yazdır.
3. `format!` ile bir `String` üret ve yazdır.

KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Hello, World!
- `std::fmt`
- `std` — `println!`

Sürüm Notları

- Rust 1.58.0 — Captured identifiers in format strings
- Rust Blog — Announcing 1.96.0

Hata Kodları

- E0277
- E0599
- E0425

Ek Kaynaklar

- Rust By Example — Formatted print
- rustc testleri — `format-args-capture`

Not: Inline capture ve width/precision capture bilgileri aramada doğrulandı. Kitap, std ve hata kodu bağlantıları standart doc.rust-lang.org adresleridir. Çıktı satırlarını ve hata mesajlarını kendi makinende bir kez çalıştırarak kontrol et.

KONU 3: Variables (let, mut)

Kategori: Temel | Sürüm: Rust 1.96.0

KATMAN 1 — İLK OKUMA

```
const MAX_POINTS: u32 = 100_000;           // const: tür yazmak zorunlu

fn main() {
    let x = 5;                               // immutable
    let mut y = 10;                          // mutable
    y += 1;                                  // OK: y değişebilir
    let x = x + 1;                           // shadowing: yeni bir x (6)

    {
        let x = x * 2;                       // iç blokta başka bir x (12)
        println!("inner x = {x}");
    }                                         // iç x burada biter

    let spaces = "  ";                       // &str
    let spaces = spaces.len();               // shadowing: tür usize oldu

    println!("x = {x}, y = {y}, spaces = {spaces}, max = {MAX_POINTS}");
}

// Çıktı:
// inner x = 12
// x = 6, y = 11, spaces = 3, max = 100000
```

Rust'ta `let` ile tanımlanan variable (değişken) varsayılan olarak immutable (değiştirilemez) olur. Değiştirmek istiyorsan mutable (değiştirilebilir) yapmak için `mut` yazman gerekir. Aynı ismi `let` ile tekrar yazarsan shadowing (gölgeleme) olur: yeni bir variable oluşur ve tür bile değişebilir. `const` ise constant (sabit) değerdir; türü yazmak zorunludur ve isim büyük harfle yazılır. Süslü parantezle açılan her block (blok) yeni bir scope (kapsam) oluşturur ve içeride yapılan shadowing blok bitince sona erer.

1.96.0 notu: Variables konusunda 1.96.0'da değişiklik yok; kurallar aynı.

KATMAN 2 — RECALL KARTLARI

• `let x = 5;` → immutable

Hata: E0384: cannot assign twice to immutable variable x (`x = 6;` yazdın)

İlgili: `mut`

• `let mut y = 10; y = 11;` → değeri değiştirir, tür aynı kalmalı

İlgili: `unused_mut` uyarısı (değiştirmiyorsan `mut` gereksiz)

• `let x = x + 1;` → shadowing: eski `x` bir kenara bırakılır, yeni variable oluşur

İlgili: `scope`

- Tür değiştirme:

```
let s = "abc";
let s = s.len();      // OK: shadowing, tür değişir
let mut t = "abc";
t = t.len();          // E0308: mismatched types
```

• `let z: i64 = 10;` → type annotation (tür belirtme); yazmazsan type inference (tür çıkarımı) devreye girer

• `const SECS: u32 = 60 * 60 * 3;` → tür zorunlu, UPPERSNAKECASE, `mut` yok

Hata: `const` ile `mut` birleşmez (`const` globals cannot be mutable)

İlgili: `static`

• `let n = 5; const C: i32 = n;` → E0435: attempt to use a non-constant value in a constant

• `let a; a = 1;` → sonradan değer verme (deferred initialization) OK

Hata: E0381: değer atamadan `a`'yı kullandın

let unused = 5; → `` öneki "kullanmıyorum" der, uyarı gelmez

Hata/uyarı: warning: unused variable

let (a, b) = (1, 2); → destructuring ile iki variable tanımlar

GoT bağlantı haritası:

```
let  mut  aynı tür, aynı variable
     shadowing  yeni variable, tür değişebilir
     tür: annotation / inference
     deferred init  E0381
const  tür zorunlu  UPPER_SNAKE_CASE  compile time değeri
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Bellekte ne oluyor?" Cevap: yerel variable'lar fonksiyonun stack frame'inde yaşar. i32 4 byte, &str 16 byte (ptr + len) tutar.
 - mut bellek düzenini değiştirmez. Immutability tamamen compile time kavramıdır ve runtime maliyeti yoktur.
 - Shadowing yeni bir variable demektir (derleyici için ayrı bir local). Eski değer scope sonuna kadar yaşar. Örneğin bir String shadow edilince hemen drop edilmez, blok bitince edilir.
 - const bir bellek adresine sahip değildir; değer kullanıldığı her yere inline edilir. Sabit adres isteyen static kullanır.
- let kontrol için: std::mem::size_of::<i32>() → 4

Adım 2 — Compiler Davranışı

- Önce soru: "Hangi kontrol yapılıyor?" Cevap: E0384 borrow checker aşamasında yakalanır; E0308 type checker aşamasında.
 - Lint'ler: unusedvariables ve unusedmut uyarı verir, hata değil.
- Tür çıkarımı: let x = 5; → varsayılan i32. Sonradan let y: u8 = x; yazarsan çıkarım u8 olur.
- MIR'de her shadow edilen variable ayrı local olarak (1, 3 gibi) görünür. Görmek için: cargo rustc -- --emit=mir.

Adım 3 — Hata Kodları

- E0384: immutable variable'a ikinci kez değer atadın. Çözüm: let mut yap veya shadowing kullan. Rust Book bu hatayı ilk örnek olarak gösterir. Detay: rustc --explain E0384.
- E0308: mut variable'ın türünü değiştirmeye çalıştın. Çözüm: shadowing kullan veya doğru türde değer ata.
- E0381: değer atanmamış variable'ı okudun. Çözüm: tanımla ve ata, ya da her kolda atama yap.
- E0435: const içinde runtime değeri (let variable'ı) kullandın. Çözüm: sadece compile time'da bilinen değer kullan.
- Kodsuz: missing type for const item → const için tür yazmadın.

Adım 4 — "Neden Böyle?"

- Çözülen problem: "Bu değer bir yerde sessizce değişti" hataları.
- C/C++'ta `int x = 5;` değişkendir, `const int` ise sonradan istenir. Rust bunu tersine çevirir: varsayılan sabit, değişim mut ile açıkça belirtilir.
- Immutable varsayılan olunca kod okurken "bu değer değişmez" garantisi alırsın; eşzamanlılıkta (concurrency) da paylaşım güvenli olur.
- Seçim: değeri aynı türde yerinde güncelleyeceksen mut, dönüştürüp yeni isimle devam edeceksen shadowing.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni Range* tipleri, `assert_matches!` makrosu, Wasm linker güncellemeleri) variables, shadowing ve const kurallarını etkilemiyor.
- LLD linker ve cargo publish --workspace değişiklikleri bu konuyu etkilemez.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: Shadowing ile mut farkı

- Neden unutuluyor: ikisi de "değer değişti" gibi görünüyor.
- Yol 1 (analoji): mut = aynı kutunun içeriğini değiştir; shadowing = yeni kutu al, eskisini kenara koy.
- Yol 2 (kod): `let s = "a"; let s = s.len();` OK; `let mut s = "a"; s = s.len();` hata.
- Yol 3 (hata): E0308: mismatched types

Sık Unutulan 2: const için tür yazmak

- Neden unutuluyor: let'te tür yazmak zorunlu değil.
- Yol 1 (görsel): const = tabelaya yazılan sabit, tür de tabelada olmalı.
- Yol 2 (kod): `const MAX: u32 = 100;`

- Yol 3 (hata): missing type for const item

Sık Unutulan 3: Shadowing scope sonunda biter

- Neden unutuluyor: iç blokta x değişmiş gibi görünür.
- Yol 1 (analoji): iç blok geçici bir maske; çıkınca yüz eski hâline döner.
- Yol 2 (kod): iç blokta `let x = 12;`, dışarıda `x` hâlâ 6.
- Yol 3 (hata): iç bloktaki `y` dışarıda kullanılırsa E0425: cannot find value `y` in this scope

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da Variables (`let`, `mut`, `shadowing`, `const`) konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: Stack'te ne var? Kaç byte? `mut` bellek düzenini değiştiriyor mu? Shadowing ile eski değere ne oluyor? `const` ile `static` farkı nedir?
2. COMPILER DAVRANIŞI: Borrow checker E0384'ü nasıl buluyor? Tür çıkarımı nasıl çalışıyor? `unused_variables` ve `unused_mut` lint'leri. MIR çıktısında shadow edilen variable'lar nasıl görünür? (`--emit=mir`)
3. HATA KODLARI: E0384, E0308, E0381, E0435. Her biri için: ne zaman, neden, nasıl çözülür. ``rustc --explain EXXX`` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: `int x`, `const int`, `constexpr` ile karşılaştır. Rust varsayılanı neden immutable yapıyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: kalemle yazı / kurşun kalem, kutular, etiketler).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. let x = 5; yaz ve yazdır. Sonra let mut yapıp x'i 10 yap ve yazdır.
2. const MAX_USERS: u32 = 50; tanımla ve yazdır.

Orta

3. let x = 5; ile başla; shadowing ile x'i x + 1 yap. Bir iç blokta x * 2 olarak tekrar shadow et. Üç değeri de yazdır.
4. let spaces = " "; sonra shadowing ile spaces.len() değerine çevir ve yazdır.

Zor

5. let a; ile tanımla, if ile iki farklı kolda a'ya değer ata, sonra yazdır. Sonra a'yı değer atamadan yazdırmayı dene ve E0381'i gör. Son olarak let (p, q) = (1, 2); ile destructuring yap.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin 

Kendini Test Et (2-3 saat sonra, sıfırdan)

1. let, let mut ve shadowing ile birer örnek yaz.
2. const tanımla (tür dahil).
3. E0384 ve E0308 hatalarını bilerek üret ve nedenini söyle.



KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Variables and Mutability
- Rust Reference — Constant items
- std — mem::size_of

Sürüm Notları

- Rust Blog — Announcing 1.96.0

Hata Kodları

- E0384
- E0308

- E0381
- E0435

Ek Kaynaklar

- Rust By Example — Variable Bindings
- Web3 Foundation — Variables & Mutability

Not: E0384 hata metni, const için tür zorunluluğu ve shadowing davranışı aramada Rust Book üzerinden doğrulandı. Diğer bağlantılar standart doc.rust-lang.org adresleridir. Kodları kendi makinende bir kez çalıştırarak kontrol et.

KONU 4: Data Types (int, float, bool, char)

Kategori: Temel | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```
fn main() {
    let a: i32 = -42;           // işaretli tam sayı
    let b: u8 = 255;           // işaretsiz, max 255
    let c = 1_000_000;         // tür yazmadık → i32
    let pi: f64 = 3.14159;     // ondalıklı (varsayılan f64)
    let half = 0.5_f32;        // tür eki ile f32
    let ok: bool = true;       // mantıksal değer
    let letter: char = 'R';    // tek tırnak → char
    let heart = '♥';           // char Unicode da tutar

    println!("{a} {b} {c}");
    println!("{pi} {half}");
    println!("{ok} {letter} {heart}");
    println!("{}", 7 / 2);      // tam sayı bölmesi
    println!("{}", 7.0 / 2.0); // ondalıklı bölme
    println!("{}", 7 % 2);      // kalan
}

// Çıktı:
// -42 255 1000000
// 3.14159 0.5
// true R ♥
// 3
// 3.5
// 1
```

Rust'ta her değer bir data type (veri tipi) vardır ve dört scalar (tekil değer) tip temeldir: integer (tam sayı), floating-point (kayan noktalı sayı), bool (mantıksal değer) ve char (karakter). Tam sayıda varsayılan tip i32, ondalıklı sayıda f64'tür. u8 gibi unsigned (işaretsiz) tipler sadece pozitif sayı tutar, i32 gibi signed (işaretli) tipler negatif de tutar. char tek tırnakla yazılır ve 4 byte yer kaplar; çift tırnak ise string olur. İki tam sayıyı bölersen sonuç da tam sayı çıkar ($7 / 2 = 3$).

1.96.0 notu: Temel tipler aynı. Float tipleri için bazı matematik fonksiyonları (floor, ceil, round) bu sürümde const olarak kullanılabilir hale geldi (aşağıda Katman 3, Adım 5).

KATMAN 2 — RECALL KARTLARI

- Tam sayı tipleri:

```
i8 i16 i32 i64 i128 isize    (işaretli)
u8 u16 u32 u64 u128 usize    (işaretsiz)
```

u8 → 0..=255, i8 → -128..=127, isize/usize → işlemci genişliği (64-bit'te 8 byte)

İlgili: usize index ve uzunluk için kullanılır

- Literal (sabit ifade) yazımı:

```
let a = 1_000;    // ayırıcı _
let b = 0xff;     // hex
let c = 0o77;     // octal
let d = 0b1010;   // binary
let e = b'A';     // u8 (65)
let f = 10u8;     // tür eki
```

Hata: let x: u8 = 256; → error: literal out of range for u8 (kod yok, deny-by-default lint)

- Overflow (taşma):

▮ debug: panic → attempt to add with overflow

▮ release: wrapping, yani $255 + 1 \rightarrow 0$

- Kontrollü yollar:

```
255u8.checked_add(1)    // None
255u8.wrapping_add(1)   // 0
255u8.saturating_add(1) // 255
255u8.overflowing_add(1) // (0, true)
```

▮ Bölme: $7 / 2 \rightarrow 3$, $-7 / 2 \rightarrow -3$ (sıfıra doğru), $7 \% 2 \rightarrow 1$

Hata (runtime): sıfıra bölme → panic attempt to divide by zero

- Float:

```
0.1 + 0.2 == 0.3        // false
let n = f64::NAN;
n.is_nan()              // true
```

Hata: $7 / 2.0 \rightarrow E0277$: cannot divide {integer} by {float}

- Tipleri karıştırma:

```
let a: i32 = 1;
let b: i64 = 2;
let c = a + b;    // E0308: mismatched types
```

- as ile dönüştürme:

```
3.9_f64 as i32      // 3 (kesilir)
-1_i32 as u32       // 4294967295
300_i32 as u8        // 44
```

- bool:

```
let t: bool = true;
!t && false || true    // mantıksal işlemler
```

Hata: if 1 {} → E0308: expected bool, found integer

- char:

```
'a' as u32          // 97
97u8 as char         // 'a'
```

Hata: 65u32 as char → E0604: only u8 can be cast as char

Hata: let c: char = "a"; → E0308 (çift tırnak = string)

- "42".parse::<i32>() → Result döner (tür belirtmeden parse() yazarsan E0284: type annotations needed)

GoT bağlantı haritası:

```
scalar [X][X][X] integer [X][X][X] signed (i8 ... i128, isize)
      [X]                [X][X] unsigned (u8 ... u128, usize)
      [X]                [X][X] overflow [X] checked_ / wrapping_ / saturating_
      [X][X] float (f32, f64) [X] NaN
      [X][X] bool [X] if / && / ||
      [X][X] char (4 byte) [X] as u32 / u8 as char
as [X] tür dönüştürme (kayıplı olabilir)
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Bellekte ne oluyor?" Cevap: bu tipler stack'te yaşar ve boyutları sabittir.

Tip	Boyut
-----	-------

i8 / u8 / bool	1 byte
i16 / u16	2 byte
i32 / u32 / f32 / char	4 byte
i64 / u64 / f64	8 byte
i128 / u128	16 byte
isize / usize	pointer genişliği (64-bit'te 8 byte)

- İşaretli sayılar two's complement (ikiye tümleyen) ile tutulur. Örneğin -i8 bellekte 0xFF'tir. Bu yüzden -i32 as u32 sonucu 4294967295 olur: bit'ler aynı, yorum farklı.
- Float'lar IEEE 754 biçimindedir: f64 için 1 bit işaret, 11 bit exponent, 52 bit mantissa. 0.1'in ikilik tabanda tam karşılığı olmadığı için $0.1 + 0.2 \neq 0.3$ çıkar.
- char bir Unicode scalar value'dur ($0..=0x10FFFF$, surrogate aralığı hariç) ve sabit 4 byte tutar. Bu yüzden 'a' 4 byte, ama "a" string'i UTF-8 olduğu için 1 byte veri tutar.
- bool 1 byte yer kaplar ve sadece 0 veya 1 değerini alabilir.
- Kontrol için: `std::mem::size_of::<char>() → 4`

Adım 2 — Compiler Davranışı

- Önce soru: "Tür yazmazsam ne olur?" Cevap: type inference, kısıt bulamazsa tam sayıya i32, ondalıklıya f64 verir.
- `let x: u8 = 256;` ve `let y: u8 = 255 + 1;` gibi compile time'da bilinen taşmalar derleme hatasıdır (overflowing literals ve arithmetic overflow lint'leri varsayılan olarak deny).
- Runtime taşması debug'da kontrol edilir. MIR'de toplama `AddWithOverflow` ve ardından bir assert olarak görünür. Release profilinde bu kontrol yoktur. Cargo.toml'da `overflow-checks = true` ile release'e de açabilirsiniz.
- as dönüşümü hata vermez, bit seviyesinde keser veya yeniden yorumlar. Float'tan tam sayıya as ise saturating'dir: `300.7_f64 as u8 → 255`, `NaN as u8 → 0`.
- MIR görmek için: `cargo rustc -- --emit=mir`.

Adım 3 — Hata Kodları

- E0308: tip uyumsuzluğu (i32 + i64, if 1, char yerine string). Çözüm: aynı tipe getir (b as i32 veya `i32::try_from(b)`).
- E0277: o tip çifti için işlem tanımlı değil (`{integer} / {float}`). Çözüm: iki tarafı da aynı türde yaz (`7.0 / 2.0`).
- E0604: sadece u8, char'a as ile çevrilebilir. Çözüm: `char::from_u32(65)` (`Option<char>` döner).
- E0284: `parse()` hangi tipe çevireceğini bilmiyor. Çözüm: `let n: u32 = ...` veya `parse::<u32>()`.
- Kodsuz: literal out of range for u8, this arithmetic operation will overflow
- Runtime panic: attempt to add with overflow, attempt to divide by zero
- Detay için: `rustc --explain E0308` (açıklama: beklenen tipin ve bulunan tipin eşleşmemesi).

Adım 4 — "Neden Böyle?"

- Çözülen problem: C'de int boyutu platforma bağlıdır ve tipler sessizce dönüşür (örneğin signed ile unsigned karşılaştırması). Rust'ta i32, u64 gibi tipler sabit genişliktedir ve dönüşüm açıktır.
- C'de signed overflow tanımsız davranıştır (undefined behavior). Rust'ta debug'da panic, release'te wrapping gibi tanımlı bir davranış vardır.
- C'nin char'ı 1 byte'tır; Rust'ın char'ı Unicode scalar value'dur.
- Trade-off: açık dönüşüm yazmak biraz daha uzun, ama hatalar derleme zamanında yakalanır.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni Range* tipleri, assert_matches! makrosu, Wasm linker güncellemeleri) scalar data type kurallarını değiştirmiyor.
- Veri tiplerinin temel kuralları (boyutlar, overflow, as) değişmedi.
- Yeni Range* tipleri scalar tiplerin temel kurallarını değiştirmiyor; assert_matches! makrosu ve Wasm linker güncellemeleri de bu konuyu etkilemiyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: Overflow debug ve release'te farklı davranır

- Neden unutuluyor: kodu debug'da çalıştırırsın ve release'te farklı sonuç beklemezsin.
- Yol 1 (analoji): kilometre sayacı, 999999'dan sonra 000000'a döner (wrapping). Debug'da ise sayaç alarm verir (panic).
- Yol 2 (kod): `255u8.checked_add(1) → None`
- Yol 3 (hata): thread 'main' panicked: attempt to add with overflow

Sık Unutulan 2: Tam sayı ve ondalıklıyı karıştırmak

- Neden unutuluyor: matematikte $7 / 2.0$ doğal görünür.
- Yol 1 (görsel): nokta var mı? $7 / 2 = 3$, $7.0 / 2.0 = 3.5$
- Yol 2 (kod): `7 as f64 / 2.0 → 3.5`
- Yol 3 (hata): E0277: cannot divide {integer} by {float}

Sık Unutulan 3: Tek tırnak mı çift tırnak mı?

- Neden unutuluyor: başka dillerde ikisi de string olabilir.
- Yol 1 (analoji): tek tırnak = tek harf (tek kişilik koltuk), çift tırnak = metin.
- Yol 2 (kod): `let c: char = 'a';` ve `let s: &str = "a";`

Yol 3 (hata): `let c: char = "a";` → E0308: mismatched types

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da Data Types (`int`, `float`, `bool`, `char`) konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: Her tip stack'te kaç byte yer kaplar? Two's complement nasıl çalışır? IEEE 754 f32/f64 bit düzeni nedir? `char` ve `bool`'un geçerli değer aralıkları nelerdir?
2. COMPILER DAVRANIŞI: Tür çıkarımı varsayılan tipleri (`i32`, `f64`) nasıl seçiyor? Overflow kontrolü MIR'de nasıl görünür? `overflowing_literals` ve `arithmetic_overflow` lint'leri. `as` dönüşümü bit seviyesinde ne yapar?
3. HATA KODLARI: E0308, E0277, E0604, E0284. Her biri için: ne zaman, neden, nasıl çözülür. ``rustc --explain EXXX`` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: `int`, `long`, `char`, signed overflow davranışı. Rust neden sabit genişlikli tipler ve açık dönüşüm kullanıyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: kutu boyutları, kilometre sayacı).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. `i32`, `u8`, `f64`, `bool` ve `char` tiplerinde birer variable tanımla (tür yazarak) ve hepsini yazdır.
2. `7 / 2`, `7 % 2` ve `7.0 / 2.0` sonuçlarını yazdır.



Orta

3. `255u8` için `checkedadd(1)`, `wrappingadd(1)` ve `saturating_add(1)` sonuçlarını yazdır.

4. 'a' karakterini u32'ye, 97u8 değerini char'a çevirip yazdır.



Zor

5. let a: i32 = 10; let b: i64 = 20; ile toplam bul: E0308'i gör, sonra as i64 ile düzelt. Ardından 3.9f64 as i32 ve -1i32 as u32 sonuçlarını yazdır.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. Beş temel tipi birer örnekle yaz.
2. u8 overflow'unu üç farklı yöntemle ele al.
3. as ile üç dönüşüm yap ve sonuçlarını tahmin et.



KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Data Types
- Rust Reference — Numeric types
- Rust Reference — Type cast expressions
- std — i32 (primitive)

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0308
- E0277
- E0604
- E0284

Ek Kaynaklar

- Rust By Example — Primitives
- Rust Book Abridged — Common Programming Concepts

Not: i32/f64 varsayılanları, overflow davranışı (debug panic, release wrapping) ve char'ın 4 byte olması aramada Rust Book üzerinden doğrulandı. as dönüşüm örnekleri, boyut tablosu ve hata kodu bağlantıları standart bilgidir; kodları kendi makinende çalıştırarak sonuçları kontrol et.

KONU 5: Operators (+, -, *, /, %, ==, &&, ||)

Kategori: Temel | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```

fn main() {
    let a = 17;
    let b = 5;

    println!("{}", a + b);           // toplama
    println!("{}", a - b);           // çıkarma
    println!("{}", a * b);           // çarpma
    println!("{}", a / b);           // tam sayı bölmesi
    println!("{}", a % b);           // kalan
    println!("{}", a == b);          // eşit mi?
    println!("{}", a != b);          // eşit değil mi?
    println!("{}", a > b && b > 0);   // ve
    println!("{}", a < b || b > 0);   // veya
    println!("{}", !(a > b));        // değil

    let mut c = 10;
    c += 5;                          // c = c + 5
    c *= 2;                          // c = c * 2
    println!("{}", c);

    println!("{}", 1 << 3);          // bit kaydırma
    println!("{}", 0b1100 & 0b1010); // bit düzeyinde ve
}

// Çıktı:
// 22
// 12
// 85
// 3
// 2
// false
// true
// true
// true
// false
// 30
// 8
// 8

```

Operator (işleç), operand (işlenen) değerler üzerinde işlem yapan sembollerdir. Aritmetik işleçler + - * / %, karşılaştırma işleçleri == != < > <= >= ve mantıksal işleçler && || ! olarak üç ana gruba ayrılır. Karşılaştırma ve mantıksal işleçler her zaman bool üretir. += gibi compound assignment (bileşik atama) işleçleri değişkeni yerinde günceller ve variable'ın mut olmasını ister. && ve || short-circuit (kısık devre) çalışır: sol taraf sonucu belirliyorsa sağ tarafa bakmaz.

1.96.0 notu: İşleç kuralları 1.96.0'da aynı.

- Aritmetik: + - * / %

```
7 / 2      // 3   (tam sayı bölmesi, sıfıra doğru kesilir)
-7 / 2     // -3
7.0 / 2.0  // 3.5
```

Hata (runtime): sıfıra bölme → panic attempt to divide by zero

- Kalan (%) işareti soldaki sayıyı izler:

```
-7 % 3      // -1
(-7_i32).rem_euclid(3) // 2
7.5 % 2.0    // 1.5 (float'ta da çalışır)
```

- Karşılaştırma: iki taraf aynı tipte olmalı

```
1 == 1.0    // E0277: can't compare `{integer}` with `{float}`
```

İlgili: PartialEq, PartialOrd

- Mantıksal: && (ve), || (veya), ! (değil), hepsi bool ister

```
v.len() > 0 && v[0] == 1 // sol false ise sağ hiç çalışmaz
```

İlgili: & ve | bool üzerinde short-circuit yapmaz

- Bit düzeyinde ve kaydırma:

```
0b1100 & 0b1010 // 8   (ve)
0b1100 | 0b1010 // 14  (veya)
0b1100 ^ 0b1010 // 6   (xor)
!0u8           // 255 (bit tersi)
1 << 3         // 8
16 >> 2        // 4
```

Hata (runtime): 1u8 << 8 → panic attempt to shift left with overflow

- Compound assignment: += -= *= /= %= &= |= ^= <<= >>=

Hata: let x = 1; x += 1; → E0384: cannot assign twice to immutable variable

Hata: x++ → error: Rust has no postfix increment operator → x += 1 yaz

- Öncelik (yüksekten düşüğe):

```

. ( ) [ ] ?      (metot, çağrı, indeks, ?)
- ! * &          (tekli işleçler)
as
* / %
+ -
<< >>
&
^
|
== != < > <= >= (zincirlenemez)
&&
||
.. ..=
= += -= ...

```

Şüphedeysen parantez kullan.

- as çarpmadan önce gelir; as sonrası < sorun çıkarır:

```

a as u64 * b          // (a as u64) * b
(a as u64) < b        // parantez şart

```

Hata: `a as u64 < b` → error: < is interpreted as a start of generic arguments for u64, not a comparison

- Zincirleme karşılaştırma yasak:

```

1 < x < 10           // error: comparison operators cannot be chained
1 < x && x < 10       // doğrusu

```

- = ifade olarak () döner, koşul bool ister:

Hata: `if x = 5 { }` → E0308: mismatched types (expected bool, found ())

- Kendi tipin için işleç:

Hata: `p1 + p2 (Point)` → E0369: binary operation + cannot be applied to type Point

İlgili: `impl Add for Point`

GoT bağlantı haritası:

```

operator  arithmetic (+ - * / %)  overflow / sıfıra bölme panic
           comparison (== != < > <= >=)  bool  if / while
           logical (&& || !)  short-circuit
           bitwise (& | ^ << >>)
           compound (+= ...)  mut gerekir
           as  tür dönüştürme (öncelik yüksek)
trait'ler: Add / PartialEq / PartialOrd  E0369

```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Bellekte ne oluyor?" Cevap: primitive tiplerde işleçler doğrudan CPU komutuna dönüşür (add, sub, imul, idiv, cmp). İşlenenler çoğunlukla register'larda yaşar, stack'e çoğu zaman gerek kalmaz.
- Debug build'de toplama sonrasına bir overflow (taşma) kontrolü eklenir (örneğin jo, yani taşma varsa atla). Release'te bu kontrol yoktur.
- i32::MIN / -1 donanımda tuzak (trap) oluşturur; Rust bunu önceden kontrol edip panic'e çevirir.
- && ve || koşullu atlama (branch) olarak derlenir; sol taraf sonucu belirliyorsa sağ tarafın kodu atlanır.
- İşaretsiz sayıda x / 8 gibi ifadeler optimizasyonla bit kaydırmaya dönüşebilir.

Adım 2 — Compiler Davranışı

- Önce soru: "Compiler a + b'yi nasıl görüyor?" Cevap: primitive tiplerde yerleşik işleç; kendi tiplerinde ise trait (özellik) çağrısı: $a + b \rightarrow \text{Add}::\text{add}(a, b)$, $a += b \rightarrow \text{AddAssign}$, $a == b \rightarrow \text{PartialEq}::\text{eq}(\&a, \&b)$, $a < b \rightarrow \text{PartialOrd}$.
- `referans alır, bu yüzden s1 s2 String'leri taşımaz. s1 + &s2 ise s1'i taşır.
- Öncelik parser aşamasında çözülür. Karşılaştırma işleçleri associative (birleşme yönü) olmadığı için zincirlenemez.
- Compile time'da bilinen ifadeler (2 + 3) constant folding ile hesaplanır.
- MIR'de aritmetik Add, Rem, Lt gibi işlemler olarak, kontrollü olanlar AddWithOverflow + assert olarak görünür. Görmek için: cargo rustc -- --emit=mir.

Adım 3 — Hata Kodları

- E0369: işleç o tip için tanımlı değil. Çözüm: ilgili trait'i implement et (impl Add for Point) veya alanlara ayrı ayrı işlem yap. Detay: rustc --explain E0369.
- E0277: tip çifti için işlem yok ({integer} ile {float} karşılaştırması). Çözüm: tipleri eşitle.
- E0308: i32 + i64 veya if x = 5. Çözüm: aynı tipe çevir / == yaz.
- E0384: mut olmayan variable'a +=. Çözüm: let mut.
- E0600: ! işleci o tipe uygulanamaz (örneğin !"text"). Çözüm: bool veya tam sayıya uygula.
- Kodsuz: comparison operators cannot be chained, Rust has no postfix increment operator, < is interpreted as a start of generic arguments
- Runtime panic: attempt to divide by zero, attempt to calculate the remainder with a divisor of zero, attempt to shift left with overflow

Adım 4 — "Neden Böyle?"

- C/C++'ta `if (x = 5)` geçerli bir koşuldur ve klasik bir hata kaynağıdır. Rust'ta koşul `bool` olmak zorunda olduğu için derlenmez.
- C'de `x++` ifadesinin değerlendirme sırası karışık davranışlara yol açabilir. Rust bu operatörü hiç sunmaz, `x += 1` kullanırsın.
- C'de `a & b` `c`, `a & (b c)` olarak ayrışır (sürpriz). Rust'ta `&` karşılaştırmadan önce gelir.
- Rust'ta tipler arası otomatik dönüşüm yok: `1 == 1.0` derlenmez.
- Trade-off: biraz daha çok yazarsın, ama anlam belirsizliği azalır.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni `Range*` tipleri, `assert_matches!` makrosu, Wasm linker güncellemeleri) operatör önceliğini ve temel overflow davranışını değiştirmiyor.
- `Range*` tiplerinin stabilize edilmesi `range` operatörleriyle ilgili API'leri etkiler; ancak diğer operatör kuralları, öncelik tablosu ve overflow davranışı değişmedi.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: `=` ile `==` karışması

- Neden unutuluyor: başka dillerde `if (x = 5)` bazen sessizce çalışır.
- Yol 1 (analoji): `=` "ata" (emir), `==` "eşit mi?" (soru).
- Yol 2 (kod): `if x == 5 { }`
- Yol 3 (hata): `if x = 5 { }` → E0308: mismatched types, expected bool, found ()

Sık Unutulan 2: `x++` yok

- Neden unutuluyor: C, Java, JavaScript alışkanlığı.
- Yol 1 (görsel): `++` yok, ama `+= 1` var.
- Yol 2 (kod): `let mut x = 0; x += 1;`
- Yol 3 (hata): `x++` → error: Rust has no postfix increment operator

Sık Unutulan 3: `&&` mi `&` mi?

- Neden unutuluyor: `bool` üzerinde ikisi de çalışır.
- Yol 1 (analoji): `&&` = önce kapıyı kontrol et, kapalıysa içeri bakma. `&` = her durumda içeri bak.
- Yol 2 (kod): `v.len() > 0 && v[0] == 1` güvenli; `&` ile yazarsan `v[0]` boş vector'de panic yapar.
- Yol 3 (hata): `index out of bounds: the len is 0 but the index is 0`

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da Operators (+, -, *, /, %, ==, &&, ||) konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: İşleçler CPU seviyesinde nasıl çalışır? Overflow kontrolü assembly'de nasıl görünür? Short-circuit branch olarak nasıl derlenir?
2. COMPILER DAVRANIŞI: `a + b`, `Add::add`'e nasıl dönüşür? `==` neden referans alır? Öncelik tablosu ve `as`'ın önceliği. MIR çıktısında işleçler nasıl görünür? (`--emit=mir`)
3. HATA KODLARI: E0369, E0277, E0308, E0384, E0600. Her biri için: ne zaman, neden, nasıl çözülür. ``rustc --explain EXXXX`` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: `x++`, `if (x = 5)`, `a & b == c`. Rust neden farklı?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: kapı kontrolü, hesap makinesi).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. İki i32 variable tanımla; + - * / % sonuçlarını yazdır.
2. `==`, `!=`, `<`, `>=` karşılaştırmalarının sonuçlarını yazdır.



Orta

3. `let mut score = 10;` ile `+=`, `-=`, `*=` kullanarak değeri değiştir ve her adımda yazdır.
4. `age >= 18 && hasid` ve `isweekend || is_holiday` ifadelerini bool variable'larıyla yaz ve yazdır.



Zor

5. `-7 / 2`, `-7 % 3` ve `(-7i32).remeuclid(3)` sonuçlarını yazdır. Sonra let `v: Vec<i32> = vec![]`; için `v.len() > 0 && v[0] == 1` ifadesini panic olmadan yazdır. Son olarak `a as u64 * b` ile `(a as u64) < b` kullan.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. Beş aritmetik işleci bir örnekle yaz.
2. `&&`, `||`, `!` ile bir koşul yaz.
3. `x += 1` yaz ve `x++` yazınca ne olduğunu söyle.

KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Appendix B: Operators and Symbols
- Rust Reference — Operator expressions
- Rust Reference — Expression precedence
- `std::ops`

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0369
- E0277
- E0308
- E0384
- E0600

Ek Kaynaklar

- Rust By Example — Primitives (operators)

- Operator expressions (kısa devre açıklaması)

Not: Bu konuda arama sonuçları zayıftı. && ve || işleçlerinin kısa devre çalıştığı ve &| yerine tercih edilmesi gerektiği bilgisi sadece ikincil bir kaynakta doğrulandı. Öncelik tablosu, hata kodları ve hata mesajları bilgime dayanıyor ve resmi Reference sayfasıyla tam karşılaştırılmadı. Öncelik tablosunu Reference'taki "Expression precedence" bölümünden, hata mesajlarını da kendi makinende çalıştırarak kontrol et.

KONU 6: String vs &str

Kategori: Temel | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```

fn main() {
    let literal: &str = "hello";           // sabit metin (borrowed)
    let mut owned: String = String::from("hello"); // heap'te büyüyeblen metin
    owned.push_str(", world");             // &str ekler
    owned.push('!');                       // tek char ekler

    let slice: &str = &owned[0..5];        // String'in bir parçası

    println!("{literal}");
    println!("{owned}");
    println!("{slice}");

    println!("{}", owned.len());           // byte sayısı
    println!("{}", "héllö".len());        // é iki byte
    println!("{}", "héllö".chars().count()); // karakter sayısı

    greet(&owned);                         // &String → &str
    greet("literal");                      // &str direkt
}

fn greet(name: &str) {
    println!("Hi, {name}!");
}

// Çıktı:
// hello
// hello, world!
// hello
// 13
// 6
// 5
// Hi, hello, world!!
// Hi, literal!

```

Rust'ta iki ana metin tipi vardır: `String` `owned` (sahipli) bir tiptir, heap'te yaşar ve büyüyebilir. `&str` ise `string slice` (metin dilimi) denen, başka bir yerdeki UTF-8 metne `borrowed` (ödünç alınmış) bir bakıştır; "hello" gibi literal'ler bu tiptedir. Bir fonksiyon sadece okuyacaksa parametresini `&str` yaz, çünkü hem `&String` hem literal kabul eder. `len()` karakter değil byte sayısını verir; "héllö" için 6 çıkar çünkü é iki byte'tır. Metnin n. karakterine `s[n]` ile erişemezsin; bunun nedenini Katman 3'te göreceksin.

1.96.0 notu: `String` ve `&str` davranışı 1.96.0'da aynı.

KATMAN 2 — RECALL KARTLARI

- Oluşturma:

```
let a: &str = "hi";           // literal
let b = String::from("hi");   // &str → String
let c = "hi".to_string();     // aynı iş
let d = "hi".to_owned();      // aynı iş
let e = String::new();        // boş
let f = String::with_capacity(32); // önceden yer ayır
```

Hata: `let s: String = "hi";` → E0308: expected String, found &str

- Ekleme:

```
let mut s = String::from("a");
s.push_str("bc"); // &str ekler
s.push('d');      // char ekler
s += "ef";        // += da &str ister
```

Hata: `s` mut değilse → E0596: cannot borrow `s` as mutable, as it is not declared as mutable

- Birleştirme:

```
let s3 = s1 + &s2;           // s1 TAŞINIR, s2 kalır
let s4 = format!("{s2}-{s3}"); // hiçbirini taşınmaz
```

Hata: `s1` sonra kullanılırsa → E0382: borrow of moved value

Hata: `"a" + "b"` → E0369: cannot add &str to &str

- Fonksiyon parametresi:

```
fn f(s: &str) {}
f("lit");      // OK
f(&owned);      // OK: &String → &str (deref coercion)
f(owned);      // E0308: expected &str, found String
```

- Uzunluk:

```
"héllo".len()           // 6 (byte)
"héllo".chars().count() // 5 (karakter)
```

- Indexleme yok:

```
let s = String::from("hello");
let c = s[0]; // E0277: the type `str` cannot be indexed by `{integer}`
```

- Dilimleme (byte aralığı):

```
&s[0..2]          // "he"
```

Hata (runtime): `&"é"[0..1]` → panic byte index 1 is not a char boundary

- Dolaşma:

```
for c in s.chars() {}           // karakterler
for b in s.bytes() {}          // byte'lar
for (i, c) in s.char_indices() {} // byte konumu + karakter
```

- Dönüşümler:

```
owned.as_str()           // String → &str
&owned[..]               // aynı
"42".parse::<i32>()       // Result<i32, _>
String::from_utf8(vec)    // Result<String, _>
```

GoT bağlantı haritası:

```
str (UTF-8 byte'lar) → &str (ptr, len) → literal ("..." → binary içinde)
                                   → slice (&s[a..b])
String (ptr, cap, len) → heap → push_str / push / +=
                                   → + / format!
                                   → &String → deref coercion → &str
len() = byte   |   chars() = karakter   → s[0] yok (E0277)
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Veri nerede?" Cevap:
- `&str` stack'te bir fat pointer (geniş pointer) tutar: (ptr, len), 64-bit'te 16 byte. Literal'in metni programın binary dosyasında, salt okunur bölümde durur ve program boyunca yaşar (`&static str`).
- `String` stack'te (ptr, cap, len) tutar, 64-bit'te 24 byte. Metnin kendisi heap'te bir buffer'dadır.
- `String::from("hello")` heap'te yer ayırır ve 5 byte'ı kopyalar. Metin büyüdükçe capacity (kapasite) dolunca buffer daha büyük yere taşınır (genelde katlanarak büyür). Bu yüzden `with_capacity` ile baştan yer ayırmak hızlıdır.
- `owned` scope sonunda drop edilir ve heap buffer serbest bırakılır.

- `&owned[0..5]` yeni bir kopya oluşturmaz, aynı buffer'a bakan yeni (ptr, len) üretir.
- Kontrol için: `std::mem::sizeof::<String>() → 24`, `std::mem::sizeof::<&str>() → 16` (64-bit sistem).

Adım 2 — Compiler Davranışı

- Önce soru: "greet(&owned) nasıl derleniyor?" Cevap: `&String` beklenen `&str`'ye deref coercion (otomatik başvuru dönüşümü) ile çevrilir; `&owned` kabaca `&owned[..]` olur.
- + işleci `String` için `add(self, s: &str) -> String` olarak tanımlıdır. Bu yüzden soldaki `String` taşınır (move), sağdaki `&String` ise `&str`'ye coercion ile çevrilir.
- Metnin UTF-8 geçerli olması garanti edilir. Dilimleme sırasında sınırın bir karakter başlangıcına denk gelip gelmediği runtime'da kontrol edilir, eşleşmezse panic olur.
- İndexleme bilerek yasaktır: UTF-8'de karakterler 1-4 byte olduğundan `s[n]` ne byte'ı ne karakteri güvenle verir ve $O(1)$ olamaz.
- MIR'de `String::from` ve `push_str` fonksiyon çağrılarını olarak görünür. Görmek için: `cargo rustc -- --emit=mir`.

Adım 3 — Hata Kodları

- E0308: `String` ile `&str` karıştı. Çözüm: `String::from(...).tostring()` veya `&s/.asstr()`.
- E0277: `s[0]` yazdın. Çözüm: `s.chars().nth(0)` veya byte aralığıyla `&s[0..1]` (dikkatli).
- E0369: iki `&str`'yi + ile topladın. Çözüm: `format!("{}",{a}{b})` veya soldaki `String` yap.
- E0382: `s1 + &s2`'den sonra `s1`'i kullandın. Çözüm: `s1.clone() + &s2` veya `format!`.
- E0596: mut olmayan `String`'e `push_str`. Çözüm: `let mut`.
- Runtime panic: byte index 1 is not a char boundary. Çözüm: `char_indices()` veya `s.get(0..1)` (`Option<&str>` döner).
- Detay için: `rustc --explain E0382`.

Adım 4 — "Neden Böyle?"

- C'de metin `char*` ve NUL ile biter: uzunluk bilinmez, taşma (buffer overflow) kolaydır. Rust'ta uzunluk her zaman (ptr, len) içinde taşınır.
- C++'taki `std::string` ve `std::string_view` ikilisi `String` ve `&str` ile benzer işi görür. Fark: Rust'ta borrow checker, dilimin sahibinden uzun yaşamasını derleme zamanında engeller.
- UTF-8 zorunluluğu her yerde aynı kodlamayı garanti eder; karşılığında karakter bazlı $O(1)$ indexleme yoktur.
- Trade-off: iki tip öğrenmek zorundasın, ama kim sahip, kim ödünç aldı sorusu tipte açıkça yazar.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni `Range*` tipleri, `assert_matches!` makrosu, Wasm linker güncellemeleri) `String` ve `&str`, UTF-8 veya deref coercion kurallarını değiştirmiyor.
- Bu üç 1.96.0 değişikliği `string slice` veya `String` kullanımını etkilemiyor.

KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: Ne zaman String, ne zaman &str?

- Neden unutuluyor: ikisi de "metin" gibi görünüyor.
- Yol 1 (analoji): String = sahip olduğun defter, &str = defterin bir sayfasına bakmak.
- Yol 2 (kod): parametre fn f(s: &str), saklayan alan struct User { name: String }.
- Yol 3 (hata): f(owned) → E0308: expected &str, found String

Sık Unutulan 2: s[0] yok

- Neden unutuluyor: Python ve JavaScript'te çalışır.
- Yol 1 (görsel): é iki byte, 😊 dört byte; "3. karakter" hangi byte?
- Yol 2 (kod): s.chars().nth(0) → Option<char>
- Yol 3 (hata): E0277: the type str cannot be indexed by {integer}

Sık Unutulan 3: + soldakini taşır

- Neden unutuluyor: sayılarda + hiçbir şeyi taşımaz.
- Yol 1 (analoji): a + &b = a'nın defterine b'yi ekleyip defteri yeni isme devretmek.
- Yol 2 (kod): taşımadan birleştirmek için format!("{a}{b}")
- Yol 3 (hata): E0382: borrow of moved value: a

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da String vs &str konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: Stack'te ne var, heap'te ne var? Kaç byte? String'in (ptr, cap, len) yapısı ve &str'nin fat pointer yapısı. String büyürken buffer nasıl yeniden ayrılıyor? Literal'ler nerede saklanıyor?
2. COMPILER DAVRANIŞI: Deref coercion nasıl çalışıyor? + işleci neden soldaki String'i taşıyor? Dilimlemede char boundary kontrolü nasıl yapılıyor? MIR çıktısında String işlemleri nasıl görünür? (--emit=mir)
3. HATA KODLARI: E0308, E0277, E0369, E0382, E0596. Her biri için: ne zaman, neden, nasıl çözülür. `rustc --explain EXXX` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: char*, std::string, std::string_view ile karşılaştır. Rust neden UTF-8 zorunlu kılıyor ve neden indexleme yok?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: defter ve sayfa, kitap ve fotokopi).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. Bir &str literal ve bir String oluştur, ikisini de yazdır.
2. let mut s = String::from("Rust"); yap; push_str(" is fun") ve push('!') ekle, yazdır.



Orta

3. fn shout(s: &str) -> String yaz (s.to_uppercase() döndürsün). Hem "hello" literal'i hem &String ile çağır.
4. "héllö" için len() ve chars().count() değerlerini yazdır.



Zor

5. `let a = String::from("Hello"); let b = String::from(" Rust"); let c = a + &b; yaz ve yazdır. Sonra a'yı yazdırmayı dene (E0382'yi gör), ardından format! ile taşımadan birleştir. Son olarak &c[0..5] ile dilimle ve "é" içeren bir metinde sınırı bozan dilimlemeyle panic!'i gör.`

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. String oluştur, büyüt ve `&str` parametrelili fonksiyona ver.
2. `len()` ile `chars().count()` farkını göster.
3. `+` ile `format!` farkını (move) göster.

KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Storing UTF-8 Encoded Text with Strings
- Rust Book — The Slice Type
- std — String
- std — str

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0308
- E0277
- E0369
- E0382
- E0596

Ek Kaynaklar

- Rust By Example — Strings

Not: String ve &str tanımları, add(self, s: &str) imzası, deref coercion ve E0277 indexleme hatası aramada Rust Book üzerinden doğrulandı. Boyutlar (24 ve 16 byte), büyüme stratejisi ve char boundary panic mesajı bilgime dayanıyor; size_of ve panic örneğini kendi makinende çalıştırarak kontrol et.

KONU 7: if / else if / else

Kategori: Kontrol | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```
fn main() {
    let number = 6;

    if number % 4 == 0 {
        println!("divisible by 4");
    } else if number % 3 == 0 {
        println!("divisible by 3");
    } else if number % 2 == 0 {
        println!("divisible by 2");
    } else {
        println!("not divisible by 4, 3, or 2");
    }

    let is_big = number > 5;
    let label = if is_big { "big" } else { "small" }; // if bir ifade
    println!("{label}");

    let temp = 25;
    if temp > 20 && temp < 30 { // iki koşul birlikte
        println!("warm");
    }
}

// Çıktı:
// divisible by 3
// big
// warm
```

if bir condition (koşul) alır ve bu koşul mutlaka bool (mantıksal değer) olmalıdır; sayıyı otomatik true ya da false yapmaz. Zincirdeki arm'lardan (kol) yalnızca ilk true olan çalışır, kalanlara bakılmaz. if bir expression (ifade) olduğu için değer üretir ve let label = if ... { ... } else { ... }; biçiminde kullanılabilir. Bu durumda her iki kolun da aynı tipte değer üretmesi gerekir. Koşulun etrafına parantez yazmak zorunda değilsin, ama süslü parantez her zaman zorunludur.

1.96.0 notu: if kuralları 1.96.0'da aynı.



KATMAN 2 — RECALL KARTLARI

- Temel yapı:

```
if cond { ... } else if cond2 { ... } else { ... }
```

İlgili: else zorunlu değil

- Sadece ilk true kol çalışır, sıra önemlidir:

```
let n = 12;  
if n % 2 == 0 { } // burada durur  
else if n % 4 == 0 { } // asla çalışmaz
```

- Koşul bool olmalı:

```
let n = 3;  
if n { } // E0308: expected `bool`, found integer  
if n != 0 { } // doğrusu
```

- Süslü parantez zorunlu, normal parantez gereksiz:

```
if (n > 5) { } // warning: unnecessary parentheses  
if n > 5 println!("{}", n); // error: expected `{`, found `println`
```

- if ifade olarak değer döndürür (kolların sonunda ; yok):

```
let x = if cond { 5 } else { 6 };
```

İlgili: ternary operatörü yok

- Kollar aynı tipte olmalı:

```
let x = if cond { 5 } else { "six" };
// E0308: `if` and `else` have incompatible types
```

- Değer alıyorsan else zorunlu:

```
let x = if cond { 5 };
// E0317: `if` may be missing an `else` clause
```

- Kolun sonuna ; koyma tuzağı:

```
let x = if cond { 5; } else { 6 };
// E0308: mismatched types (ilk kol () döndürdü)
```

- Birleşik koşul: &&, ||, !

```
if a > 0 && (b > 0 || c > 0) { }
```

İlgili: kısa devre (Konu 5)

- Çok else if varsa match kullan (Konu 8); desen eşleştirme için if let (Konu 18, 20)

GoT bağlantı haritası:

```
if XXXX koşul: bool XX && || ! XX E0308 (bool değilse)
    XX else if XX ilk true olan çalışır
    XX else XX değer alıyorsan zorunlu (E0317)
    XX ifade XX let x = if ... XX kollar aynı tip (E0308)
çok kol XX match (Konu 8) ; desen XX if let (Konu 18, 20)
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Bellekte ne oluyor?" Cevap: if için ayrı bir veri yapısı yoktur. Koşul bir karşılaştırma (cmp) komutuna ve ardından koşullu atlamaya (jne, jle gibi) dönüşür.
- let x = if c { 5 } else { 6 }; ifadesinde sonuç tek bir yerde (register ya da stack slot) tutulur. Her iki kolun tipi aynı olmak zorunda olmasının sebebi budur: x'in boyutu compile time'da bilinmelidir.
- Çalışmayan kolun kodu yürütülmez; bu yüzden yan etkileri de (örneğin println!) görünmez.
- Basit durumlarda derleyici dal yerine cmov gibi koşullu seçim komutu üretebilir.

Adım 2 — Compiler Davranışı

- Önce soru: "Compiler ne kontrol ediyor?" Cevap: koşulun tipinin bool olması ve kolların tip birleştirmesi (unification).
- Değer kullanılmadığında if kolları () döndürür. Bu yüzden else'siz if'in gövdesi () olmak zorundadır.
- return, break, panic! gibi hiç dönmeyen ifadelerin tipi !'dir ve herhangi bir tipe uyar:

```
let x = if c { 5 } else { return; };
```

- Sabit koşullarda (if true) ölü kod elenir.
- MIR'de if bir switchInt olarak görünür. Görmek için: cargo rustc -- --emit=mir.

Adım 3 — Hata Kodları

- E0308 (bool bekleniyor): koşul bool değil. Çözüm: $n \neq 0$ gibi karşılaştırma yaz.
- E0308 (if and else have incompatible types): kollar farklı tipte. Çözüm: ikisini aynı tipe getir.
- E0308 (mismatched types, ; yüzünden): kolun sonundaki ; değeri () yaptı. Çözüm: ;'yi kaldır.
- E0317: değer üreten if'te else yok. Çözüm: else ekle.
- Kodsuz hata: expected {, found ... (süslü parantez eksik)
- Uyarı: unnecessary parentheses around if condition
- Detay için: rustc --explain E0317.

Adım 4 — "Neden Böyle?"

- C/C++'ta if (x) tam sayıyla çalışır ve if (x = 5) geçerlidir. Rust bunu bool zorunluluğuyla engeller.
- C'de süslü parantezsiz if yazılabilir; yanlış girinti kritik güvenlik hatalarına yol açmıştır. Rust'ta süslü parantez zorunludur.
- C'deki cond ? a : b yerine Rust'ta if zaten bir ifadedir; ayrı bir ternary operatörüne gerek yoktur.
- Trade-off: biraz daha uzun yazarsın, ama anlam netleşir.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni Range* tipleri, assert_matches! makrosu, Wasm linker güncellemeleri) temel if / else if / else davranışını değiştirmiyor.
- Yakın dönemde if ile ilgili asıl yenilik let chains'tir (if let Some(x) = a && x > 5). Bu özelliğin Rust 1.88'de edition 2024 için stabilize edildiğini hatırlıyorum, ama bu oturumda doğrulamadım. Henüz öğrenmediğin bir konu olduğu için şimdilik atla.
- Bu değişiklikler if koşullarının değer üretme ve bool değerlendirme kurallarını etkilemiyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: Koşul bool olmak zorunda

- Neden unutuluyor: C, Python, JavaScript'te sayı veya metin koşul olabilir.
- Yol 1 (analoji): Rust'ta kapıcı sadece "evet" ya da "hayır" cevabını kabul eder, "3" demek yetmez.
- Yol 2 (kod): `if n != 0 { }`
- Yol 3 (hata): `if n { }` → E0308: expected bool, found integer

Sık Unutulan 2: if ifade olarak kullanılırken ; ve tip

- Neden unutuluyor: kolun son satırına alışkanlıkla ; koyarsın.
- Yol 1 (görsel): son satırda noktalı virgül = "değer üretme" demek.
- Yol 2 (kod): `let x = if c { 5 } else { 6 };` ve kolların içinde 5 yazıyor, 5; değil.
- Yol 3 (hata): E0308 ve E0317

Sık Unutulan 3: Sadece ilk true kol çalışır

- Neden unutuluyor: tüm koşulların kontrol edildiğini sanırsın.
- Yol 1 (analoji): bankodaki ilk boş gişeye gidersin, diğerlerine bakmazsın.
- Yol 2 (kod): `n = 12` için `n % 2 == 0` önce yazılırsa `n % 4 == 0` kolu asla çalışmaz; özel durumu başa yaz.
- Yol 3 (hata): derleyici hata vermez; sonuç mantıksal olarak yanlış çıkar.

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da if / else if / else konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: if assembly'de nasıl görünür? `let x = if ... { } else { }` ifadesinde sonuç nerede tutulur? Neden kolların tipi aynı olmalı?
2. COMPILER DAVRANIŞI: Koşulun bool kontrolü ve kolların tip birleştirmesi nasıl yapılıyor? Never type (!) kolu nasıl uyuyor? MIR'de switchInt nasıl görünür? (`--emit=mir`)
3. HATA KODLARI: E0308 (üç farklı durum) ve E0317. Her biri için: ne zaman, neden, nasıl çözülür. ``rustc --explain EXXX`` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: if (x), ternary operatörü, süslü parantezsiz if. Rust neden farklı?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: yol ayrımı, gişe kuyruğu).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı? (let chains dahil)
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.

GÖREV (Boş dosyaya yaz)

Kolay

1. `let number = 7;` için `number > 5` ise "big", değilse "small" yazdır (if / else).
2. `score` değişkenine göre not yazdır: `>= 90` → "A", `>= 80` → "B", diğerleri → "C" (else if zinciri).

Orta

3. `let label = if number > 5 { "big" } else { "small" };` yaz ve label'ı yazdır.
4. Bir sayının çift mi tek mi olduğunu % ile bulup yazdır.

Zor

5. `let n = 15;` için FizzBuzz mantığı yaz (önce `n % 15`, sonra `n % 3`, `n % 5`). Sonra bilerek E0317 (else'siz değer alan if) ve E0308 (`if { 5 } else { "six" }`) hatalarını üretip mesajları oku.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. if / else if / else zinciri yaz.
2. let x = if ... { } else { }; yaz.
3. if 1 { } neden hata verir, söyle.

KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Control Flow
- Rust Reference — if expressions

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0308
- E0317

Ek Kaynaklar

- Rust By Example — if/else
- Comprehensive Rust — if let, let else, while let

Not: Koşulun bool olması, else if zincirinde ilk true kolun çalışması, if'in ifade olarak kullanılması ve E0308 ("if and else have incompatible types") aramada Rust Book üzerinden doğrulandı. E0317, ; tuzağı, MIR switchInt ve let chains'in 1.88'de stabilize edildiği bilgisi bilgime dayanıyor; hata mesajlarını kendi makinende çalıştırarak kontrol et.

KONU 8: match

Kategori: Kontrol | Sürüm: Rust 1.96.0

KATMAN 1 — İLK OKUMA

```
fn main() {
    let number = 3;

    match number {
        1 => println!("one"),
        2 | 3 => println!("two or three"), // | = veya
        4..=9 => println!("four to nine"), // aralık (9 dahil)
        _ => println!("something else"), // geri kalan her şey
    }

    let size = match number { // match bir ifade
        0 => "zero",
        n if n < 0 => "negative", // guard: ek koşul
        1..=5 => "small",
        _ => "large",
    };
    println!("{size}");

    let pair = (0, -2);
    match pair {
        (0, y) => println!("x is zero, y = {y}"), // tuple parçalama
        (x, 0) => println!("y is zero, x = {x}"),
        _ => println!("no zeros"),
    }
}

// Çıktı:
// two or three
// small
// x is zero, y = -2
```

`match`, bir değeri sırayla `pattern`'lerle (desen) karşılaştırır ve eşleşen arm'ın (kol) kodunu çalıştırır. İlk eşleşen kol seçilir ve kalanlara bakılmaz. `match` exhaustive (eksiksiz) olmak zorundadır: değerin alabileceği her ihtimali karşılamalısın. Geri kalan her şeyi `_ wildcard`'ı (joker) yakalar. `match` de `if` gibi bir `expression` (ifade) olduğu için `let size = match ... { ... };` biçiminde değer üretebilir ve tüm kolların tipi aynı olmalıdır.

1.96.0 notu: `match` davranışı 1.96.0'da aynı.

KATMAN 2 — RECALL KARTLARI

- Temel yapı, ilk eşleşen kol çalışır:

```
match value {
    pattern1 => expr1,
    pattern2 => { expr_a; expr_b }    // birden fazla satır → süslü parantez
    _ => expr3,
}
```

İlgili: kollar , ile ayrılır

- Eksiksizlik, _joker:

```
let n: u8 = 5;
match n { 0 => {}, 1 => {} }
// E0004: non-exhaustive patterns: `2_u8..=u8::MAX` not covered
```

Çözüm: sona _ => {} ekle

- Birden fazla desen: 1 | 2 | 3 => ...
- Aralık: 1..=5 (5 dahil), 'a'..'z'

İlgili: a..b (b hariç) desenlerde Rust 1.80'den beri kullanılabilir

- Değeri bir isme bağlama (binding):

```
match n {
    x @ 1..=5 => println!("small {x}"),    // @ ile hem aralık hem isim
    other => println!("other {other}"),    // her şeyi yakalar
}
```

- Guard (if koşulu):

```
match n {
    x if x < 0 => "negative",
    0 => "zero",
    _ => "positive",
}
```

Hata: guard'lar eksiksizlik hesabına katılmaz:

```
match n { x if x < 0 => 1, x if x >= 0 => 2 }
// E0004: non-exhaustive patterns: `_` not covered
```

- match ifade olarak, tüm kollar aynı tipte:

```
let s = match n { 10 => 8, _ => "Not ten" };
// E0308: `match` arms have incompatible types
```

- Tuple ve bool eşleştirme:

```
match (a, b) { (true, true) => {}, (true, false) => {}, (false, _) => {} }
```

- Joker en başa konursa:

```
match n { _ => {}, 1 => {} }
// warning: unreachable pattern
```

- Kollarda farklı isimler:

```
match n { 1 | m => {} }
// E0408: variable `m` is not bound in all patterns
```

GoT bağlantı haritası:

```
match desenler: literal | aralık | tuple | _ | isim | @
    guard (if) eksiksizliğe sayılmaz
    eksiksiz olmalı E0004 _ ile çöz
    ifade kollar aynı tip (E0308)
çok kol if / else if zincirinden daha net
enum ve Option ile birlikte Konu 17, 18, 20
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Bellekte ne oluyor?" Cevap: match için ayrı bir veri yapısı yoktur. Değerler karşılaştırma ve atlama komutlarına dönüşür.
- Yoğun tam sayı kolları (0, 1, 2, ...) çoğunlukla bir jump table (atlama tablosu) olarak derlenir. Aralıklar karşılaştırma zincirine dönüşür.
- Kazanan kolun sonucu, if ifadesindeki gibi tek bir yerde (register ya da stack slot) tutulur. Bu yüzden tüm kolların tipi aynı olmalıdır.
- Binding'ler değeri kopyalar veya taşır: Copy olan tiplerde (sayılar) kopya olur, String gibi tiplerde değer bağlamaya taşınır. Taşımak istemezsen match &s { ... } yaz.

Adım 2 — Compiler Davranışı

- Önce soru: "Eksiksizliği kim kontrol ediyor?" Cevap: derleyici, desenleri bir matris gibi analiz eden bir algoritma çalıştırır ve kapsanmayan değer aralıklarını bulur. Bu yüzden hata mesajı eksik olan aralığı da gösterir (2_u8..=u8::MAX).
- Guard'lar derleyici için opak ifadelerdir; içeriklerine bakılmaz. Bu yüzden `x if x >= 0` gibi bir kol eksiksizliğe katkı yapmaz.
- Kendinden önceki desenlerce zaten kapsanan bir kol için unreachable pattern uyarısı verir.
- Kollar tip olarak birleştirilir. `return`, `panic!`, `break` gibi dönmeyen kolların tipi `!`'dir ve her tipe uyar.
- MIR'de `match`, bir decision tree (karar ağacı) olarak `switchInt` bloklarına çevrilir. Görmek için: `cargo rustc -- --emit=mir`.

Adım 3 — Hata Kodları

- E0004: `match` tüm ihtimalleri kapsamıyor. Çözüm: eksik değerleri yaz veya `_ => ...` ekle. Detay: `rustc --explain E0004`.
- E0308 (match arms have incompatible types): kollar farklı tip üretiyor. Çözüm: tipleri eşitle.
- E0408: `|` ile bağlanan değişken her desende yok. Çözüm: aynı isimleri her desende kullan.
- Uyarı: unreachable pattern, unused variable
- Not: bir enum varyantı eksikse aynı E0004 çıkar (Konu 17'de göreceksin).

Adım 4 — "Neden Böyle?"

- C'nin `switch`'inde fallthrough (alta düşme) vardır: `break` unutulursa bir sonraki kol da çalışır. Rust'ta kollar birbirinden bağımsızdır, `break` yoktur.
- C'de eksik case sessizce geçilir. Rust'ta eksiksizlik zorunlu olduğu için yeni bir değer eklendiğinde derleyici eksik yerleri gösterir.
- `match` bir ifade olduğundan her kol değer üretir; geçici değişken ya da ayrı atamalara gerek kalmaz.
- Trade-off: `_` her şeyi yakaladığı için, yeni bir değer eklendiğinde derleyicinin sana yardım etmesini engeller. Mümkünse değerleri tek tek yaz.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler içinde stabilize edilen `assert_matches!` makrosu desen eşleşmesini kontrol etmek için yeni bir araç sağlıyor; `match` ve `pattern` kuralları değişmedi.
- Aralık desenlerinde `a..b` biçimi 1.80'den beri stabil; yeni bir şey değil.
- Yeni `Range*` tipleri ve Wasm linker güncellemeleri de `match` sözdizimi ve `pattern` kurallarını değiştirmiyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: `_` kolunu unutmak

- Neden unutuluyor: `if` zincirinde `else` olmadan da yazabilirdin.
- Yol 1 (analoji): `_` = "başka ne gelirse" kutusu, onsuз sınav sorusu cevapsız kalır.
- Yol 2 (kod): `match n { 1 => ..., _ => ... }`
- Yol 3 (hata): E0004: non-exhaustive patterns: ... not covered

Sık Unutulan 2: Guard eksiksizliğe sayılmaz

- Neden unutuluyor: `x if x < 0` ve `x if x >= 0` mantıksal olarak her şeyi kapsıyor gibi görünür.
- Yol 1 (analoji): derleyici kapıdaki "özel şart" kağıtlarını okumaz, sadece desene bakar.
- Yol 2 (kod): son kolu guard'sız yaz: `_ => ...` veya `x => ...`
- Yol 3 (hata): E0004: non-exhaustive patterns: _ not covered

Sık Unutulan 3: Kollar aynı tipte olmalı, kol sonları ,

- Neden unutuluyor: `if` ile aynı kural olsa da `match`'te kol sayısı fazla olunca gözden kaçır.
- Yol 1 (görsel): her `=>` sonrası aynı tip ve sonunda virgül.
- Yol 2 (kod): `let s = match n { 0 => "zero", _ => "other" };`
- Yol 3 (hata): E0308: match arms have incompatible types

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da match konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: match assembly'de nasıl görünür (jump table, karşılaştırma zinciri)? Binding'ler değeri kopyalar mı taşır mı? match &s ile match s farkı nedir?
2. COMPILER DAVRANIŞI: Eksiksizlik kontrolü nasıl çalışıyor? Guard'lar neden hesaba katılmıyor? unreachable pattern uyarısı. MIR'de decision tree ve switchInt nasıl görünür? (--emit=mir)
3. HATA KODLARI: E0004, E0308 (match arms), E0408. Her biri için: ne zaman, neden, nasıl çözülür. `rustc --explain EXXX` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: switch, break ve fallthrough. Rust neden eksiksizlik zorunlu kılıyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: posta ayırma, yol ayrımı levhası).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.

GÖREV (Boş dosyaya yaz)

Kolay

1. let day = 3; için 1, 2, 3 kollarını ve _ kolunu yazıp bir mesaj yazdır.
2. 1 | 2 ve 4..=9 desenlerini kullanan bir match yaz.

Orta

3. let size = match n { 0 => "zero", 1..=5 => "small", _ => "large" }; yaz ve size'ı yazdır.
4. Guard ile negatif, sıfır ve pozitif ayrımı yap: x if x < 0, 0, _.

Zor

5. let pair = (0, -2); için (0, y), (x, 0), ` kollarını yaz. Sonra bilerek E0004 (eksik ) ve E0308 (match arms have incompatible types) hatalarını üretip mesajları oku. Son olarak x @ 1..=5` desenini kullan.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. _ kolu olan bir match yaz.
 2. match'i bir let ile değer döndürecek şekilde kullan.
 3. E0004'ün nedenini ve çözümünü söyle.
-



KAYNAKÇA

Resmi Kaynaklar

- Rust Book — The match Control Flow Construct
- Rust Book — Pattern Syntax
- Rust Reference — match expressions
- Rust Reference — Patterns

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0004
- E0308
- E0408

Ek Kaynaklar

- Rust By Example — match
- DEV Community — Rust match guards and exhaustive conditions

Not: match'in eksiksiz olması, _ joker kolu, ilk eşleşen kolda durması, kolların aynı tipte olması (E0308), E0004 mesajı ve guard'ların eksiksizliğe katılmaması aramada doğrulandı. Jump table, E0408, MIR switchInt ve a..b desenlerinin 1.80'den beri stabil olması bilgime dayanıyor; mesajları kendi makinende çalıştırarak kontrol et.

KONU 9: loop / while / for

Kategori: Kontrol | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```

fn main() {
    let mut count = 0;
    let result = loop {                // loop: sonsuz, break değer döndürebilir
        count += 1;
        if count == 5 {
            break count * 2;          // break ile değer çıkar
        }
    };
    println!("result = {result}");

    let mut n = 3;
    while n != 0 {                     // while: koşul doğruyken döner
        println!("{n}!");
        n -= 1;
    }
    println!("LIFTOFF!");

    let arr = [10, 20, 30];
    for x in arr {                    // for: iterator üzerinde döner
        println!("value = {x}");
    }

    for i in 0..3 {                   // Range: 0, 1, 2 (3 hariç)
        println!("i = {i}");
    }
}

// Çıktı:
// result = 10
// 3!
// 2!
// 1!
// LIFTOFF!
// value = 10
// value = 20
// value = 30
// i = 0
// i = 1
// i = 2

```

Rust'ta üç loop (döngü) yapısı vardır. loop sonsuz döner, çıkmak için break gerekir; break bir değer taşıyabilir ve bu durumda loop bir expression (ifade) olur. while her turdan önce bir condition (koşul) kontrol eder, koşul false olunca durur. for bir iterator (yineleyici) üzerinde gezer; dizi, Vec veya 0..3 gibi bir Range (aralık) üzerinde çalışır ve en güvenli, en çok kullanılan döngüdür çünkü indeks hatası yapma riski yoktur.

1.96.0 notu: loop/while/for davranışı 1.96.0'da aynı.

`loop { ... break; }` → sonsuz döngü, break olmadan asla durmaz

İlgili: `while true { }` ile aynı iş ama loop niyeti daha nettir

`break` değeri → sadece loop'ta çalışır:

```
let r = loop { break 7; }; // r = 7
```

Hata: while veya for'da break değeri → E0571: break with value from a while loop

`continue` → o turu atlar, bir sonraki tura geçer

```
for i in 0..5 {
    if i % 2 == 0 { continue; }
    println!("{i}"); // 1, 3
}
```

- while koşulu bool olmalı:

```
while n { } // E0308: expected bool, found integer
```

- Label (etiket) ile iç içe döngüden çıkma:

```
'outer: for i in 0..3 {
    for j in 0..3 {
        if j == 1 { continue 'outer; }
        if i == 2 { break 'outer; }
        println!("{i},{j}");
    }
}
```

İlgili: etiketler 'isim: ile başlar, break/continue içindeki değişkenlerden ayrı bir isim alanıdır

- for ile Range:

```
for i in 0..5 { } // 0..4 (5 hariç)
for i in 0..=5 { } // 0..5 (5 dahil)
for i in (0..5).rev() { } // ters sırada: 4,3,2,1,0
```

- for iterator'ı tüketir, in sonrası değer taşınır (Copy değilse):

```
let v = vec![1, 2, 3];
for x in v { } // v taşındı
for x in &v { } // v kalır, x: &i32
for x in v.iter() { } // &v ile aynı
```

Hata: for sonrası v kullanılırsa → E0382: borrow of moved value

- `` ile değeri kullanmamak: `for i in 0..3 { }`

- while let (desen eşleşirken döner — Konu 18, 20'de detaylandırılacak):

```
let mut stack = vec![1, 2, 3];
while let Some(top) = stack.pop() { println!("{top}"); }
```

GoT bağlantı haritası:

```
loop  break  değer döndürebilir (break val)
 sonsuz, koşulsuz
while  koşul: bool
 break değer YOK (E0571)
for  iterator üzerinde  Range (0..n, 0..=n)
 in v / in &v / in v.iter()  move / borrow
 en güvenli, indeks hatası yok
ortak: break / continue / label ('isim:)
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Bellekte ne oluyor?" Cevap: üç döngü de assembly'de bir geri-atlama (jump) ve koşullu çıkışa dönüşür; ek bir veri yapısı yoktur.
- for x in collection aslında IntoIterator::into_iter(collection) çağırır ve dönen iterator üzerinde tekrar tekrar next() çağırır; Some(x) geldikçe gövde çalışır, None gelince döngü biter. Bu yüzden for aslında bir loop + match olarak genişler (syntactic sugar — sözdizimsel şeker).
- for x in v yazarsan v'nin sahipliği into_iter()'a geçer (taşınır); for x in &v yazarsan sadece referans alınır, v scope sonunda hâlâ geçerlidir.
- loop içindeki break val ifadesinde val'in bellekteki yeri, if'teki gibi tek bir sonuç slotudur.

Adım 2 — Compiler Davranışı

- Önce soru: "for derleyiciye nasıl görünür?" Cevap: desugar (çözülme) sonrası şuna benzer:

```
let mut iter = IntoIterator::into_iter(collection);
loop {
    match iter.next() {
        Some(x) => { /* gövde */ }
        None => break,
    }
}
```

- loop'un tipi, içindeki tüm break val ifadelerinin tipinden çıkarılır; hiç break val yoksa (veya hiç break yoksa) loop'un tipi ! (never type) olur ve bu da onu her yerde kullanılabilir yapar.

- while ve for'un tipi her zaman ()'tir, çünkü gövdenin hiç çalışmama ihtimali vardır; bu yüzden break val kabul etmezler (E0571).
- Etiketler ('outer) lifetime (yaşam süresi) etiketleriyle aynı sözdizimini kullanır ama farklı bir isim alanındadır; birbirine karışmaz.
- MIR'de her üç döngü de benzer bir loop bloğu + SwitchInt + geri sıçrama olarak görünür. Görmek için: cargo rustc -- --emit=mir.

Adım 3 — Hata Kodları

- E0571: while/for içinde break değer;. Çözüm: loop kullan veya değer için ayrı bir mut değişken tut.
- E0308: while koşulu bool değil. Çözüm: karşılaştırma yaz (n != 0).
- E0382: for x in v sonrası v'yi tekrar kullandın. Çözüm: for x in &v kullan.
- E0499 / E0502 (ileride göreceksin): döngü içinde aynı anda birden fazla mutable borrow almaya çalışmak.
- Derleyici uyarısı: unused_labels, kullanılmayan 'outer etiketi.
- Detay için: rustc --explain E0571.

Adım 4 — "Neden Böyle?"

- C'de for (int i = 0; i < n; i++) üç ayrı ifade elle yazılır; sınır hatası (<= yerine < unutmak) klasik bir bug kaynağıdır. Rust'ın for x in 0..n yapısı bu sınır hesabını Range'e devreder.
- C'de do { } while vardır; Rust'ta bunun yerine loop { ...; if !cond { break; } } yazılır — ayrı bir sözdizimine gerek yoktur.
- break'in değer taşıyabilmesi, döngü sonucunu geçici bir mut değişkene atama ihtiyacını ortadan kaldırır.
- Label'lı break/continue, C'deki goto'ya ihtiyaç duymadan iç içe döngülerden çıkmayı sağlar.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki Range* tiplerinin stabilize edilmesi range tabanlı iteration API'lerini etkiliyor; ancak loop/while/for sözdizimi ve label kuralları değişmedi.
- while let zincirleri (while let Some(x) = a && x > 5 gibi) edition 2024 ile ilgili bir yenilik alanıdır; bu konunun temel while/for kurallarını etkilemez ve henüz öğrenmediğin bir konu olduğu için şimdilik atla.
- assert_matches! makrosu ve Wasm linker güncellemeleri de temel loop/while/for davranışını etkilemiyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: break val sadece loop'ta çalışır

- Neden unutuluyor: üçü de "döngü" olduğu için aynı kurallara sahip sanılır.
- Yol 1 (analoji): sadece loop bir "sonuç kutusu" sunar; while/for sadece "yap ve çık" der.
- Yol 2 (kod): `let r = loop { break 5; };`
- Yol 3 (hata): `while true { break 5; } → E0571: break with value from a while loop`

Sık Unutulan 2: for v ile for &v farkı

- Neden unutuluyor: döngüden sonra v'yi tekrar kullanmak isteyebilirsin.
- Yol 1 (analoji): `for x in v` = kutuyu devretmek; `for x in &v` = kutuya bakmak, geri vermek.
- Yol 2 (kod): `for x in &v { println!("{}", x); } println!("{}", v);`
- Yol 3 (hata): `for x in v { } println!("{}", v); → E0382: borrow of moved value: v`

Sık Unutulan 3: Label'ı unutup yanlış döngüden çıkmak

- Neden unutuluyor: `break` varsayılan olarak sadece en yakın (innermost) döngüyü keser.
- Yol 1 (görsel): etiketsiz `break` sadece "bir alt kat" çıkar, bütün binadan çıkmaz.
- Yol 2 (kod): `'outer: for i in 0..3 { for j in 0..3 { if i == 1 { break 'outer; } } }`
- Yol 3 (hata): derleyici hata vermez; sadece istediğin döngüden çıkmamış olursun (mantık hatası).

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da loop / while / for konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: Üç döngü de assembly'de nasıl görünür? break val'in sonucu nerede tutulur? for x in v ile for x in &v arasında bellekte ne fark var?
2. COMPILER DAVRANIŞI: for döngüsü IntoIterator/Iterator::next() olarak nasıl desugar oluyor? loop'un tipi break ifadelerinden nasıl çıkarılıyor (never type dahil)? Etiketlerin isim alanı nasıl çalışıyor? MIR'de loop blokları nasıl görünür? (--emit=mir)
3. HATA KODLARI: E0571, E0308, E0382. Her biri için: ne zaman, neden, nasıl çözülür. `rustc --explain EXXX` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: for(;;), do-while, goto. Rust'ın loop+break val ve label'lı break/continue yapısı bunların yerini nasıl alıyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: koşu pisti, asansör katları).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. loop ile count'u 0'dan 5'e kadar artır, 5 olunca break ile çık ve yazdır.
2. while ile 3'ten 1'e geri sayım yap ve her adımda yazdır.



Orta

3. let arr = [1, 2, 3, 4, 5]; için for ile dolaş, sadece çift sayıları yazdır (continue kullan).
4. let result = loop { ... break sayı * 2; }; yaz; sayı'yı döngü içinde artırıp bir koşulda break ile iki katını döndür.



Zor

5. İç içe iki for döngüsü yaz; 'outer: etiketini koy. i j olduğunda continue 'outer, i 3 olduğunda break 'outer yap. Sonra let v = vec![1,2,3]; for x in v { } sonrası v'yi yazdırmayı dene, E0382'yi gör ve &v ile düzelt.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. loop + break değeri ile bir sonuç üret.
2. for ile bir Range üzerinde dolaş.
3. Label'lı break/continue örneği yaz.



KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Repetition with Loops
- Rust Reference — Loops and other breakable expressions
- std::iter::Iterator

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0571
- E0308
- E0382

Ek Kaynaklar

- Rust By Example — loop, nesting and labels
- Luis Llamas — Rust loops: loop, while, for

Not: `break` `val`'in sadece `loop`'ta çalıştığı, `label`'lı `break/continue` davranışı ve `for`'un `iterator` üzerinde çalıştığı aramada `Reference` ve ek kaynaklar üzerinden doğrulandı. `for`'un `desugar` edilmiş hâli, `MIR` görünümü ve hata kodları (`E0571`, `E0382`) bilgime dayanıyor; mesajları kendi makinende çalıştırarak kontrol et.

KONU 10: Functions (fn, params, return)

Kategori: Fonksiyon | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```
fn main() {
    greet("Ferris");                // çağrı: fonksiyon main'den SONRA tanımlı olsa da
    çalışır
    let sum = add(3, 4);
    println!("sum = {sum}");
    println!("doubled = {}", double(sum));
}

fn greet(name: &str) {              // parametre tipi zorunlu
    println!("Hello, {name}!");
}

fn add(a: i32, b: i32) -> i32 {      // -> dönüş tipi
    a + b                           // son ifade, noktalı virgülsüz → dönüş değeri
}

fn double(x: i32) -> i32 {
    return x * 2;                   // return da kullanılabilir (erken çıkış için
    tipik)
}

// Çıktı:
// Hello, Ferris!
// sum = 7
// doubled = 14
```

Rust'ta fonksiyonlar `fn` ile tanımlanır ve her parametrenin tipi açıkça yazılmak zorundadır; Rust bunu çıkaramaz. Fonksiyonlar dosyada herhangi bir sırada tanımlanabilir, `main`'den önce ya da sonra olması önemli değildir — önemli olan aynı `scope`'ta (kapsam) görünür olmasıdır. Bir fonksiyonun gövdesi bir `block` (blok) `expression`'dır (ifade); son satır noktalı virgülsüz bırakılırsa o satırın değeri otomatik dönüş değeri olur. `return` anahtar kelimesi genelde fonksiyonun ortasından erken çıkmak için kullanılır.

1.96.0 notu: Fonksiyon tanımlama kuralları 1.96.0'da aynı.

KATMAN 2 — RECALL KARTLARI

- Temel imza: `fn isim(param: Tip, param2: Tip2) -> DönüşTipi { ... }`

İlgili: dönüş tipi yoksa `()` (unit) varsayılır

- Parametre tipi zorunlu:

```
fn f(x) { } // error: expected one of `:`, found ``
fn f(x: i32) { } // doğrusu
```

- Son ifade = dönüş değeri (noktalı virgülsüz):

```
fn add(a: i32, b: i32) -> i32 {
    a + b // dönüş değeri
}
```

Hata: sona ; koyarsan:

```
fn add(a: i32, b: i32) -> i32 {
    a + b; // artık bir statement, değer üretmiyor
}
// E0308: mismatched types, expected i32, found ()
```

- Erken çıkış: `return` değeri;

```
fn abs(x: i32) -> i32 {
    if x < 0 { return -x; }
    x
}
```

- Dönüş tipi `()` (hiçbir şey döndürmeyen fonksiyon):

```
fn log(msg: &str) { println!("{msg}"); } // -> () yazmaya gerek yok
```

Hata: gövdede değer varsa ama -> yoksa:

```
fn f() { 5 }
// E0308: mismatched types, expected `()`, found integer
```

- Fonksiyon sırası önemli değil (hoisting benzeri davranış):

```
fn main() { helper(); }
fn helper() { println!("called"); } // main'den sonra tanımlı, sorun yok
```

- Parametre ile argument (argüman) farkı: tanımdaki x: i32 parametre, çağrıdaki 5 argümandır

- Statement (ifade dizisi, değer üretmez) vs expression (değer üretir):

```
let y = (let x = 5); // error: `let` ifadedeği, statement'tır
let z = { let a = 3; a + 1 }; // z = 4, blok bir expression
```

- Fonksiyon pointer'ı (ileri konularda detaylanacak):

```
let f: fn(i32, i32) -> i32 = add;
```

GoT bağlantı haritası:

```
fn isim(param: Tip) -> DönüşTip  gövde = block expression
                                 son ifade (;'siz)  return değeri
                                 return değer;  erken çıkış
                                 -> yoksa  () (unit)
statement (;'li) vs expression (;'siz)  E0308 kaynağı
tanım sırası önemsiz  aynı scope yeterli
```



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Fonksiyon çağrılınca bellekte ne oluyor?" Cevap: her çağrı, çağıran fonksiyonun üstüne yeni bir stack frame (yığın çerçevesi) iter. Parametreler bu frame'e kopyalanır (Copy tiplerde) veya taşınır (owned tiplerde, örneğin String).
- Dönüş değeri, çağıranın beklediği yere (genelde bir register, örn. x86-64'te rax) yazılır. Büyük struct'lar dönülürken derleyici gizli bir pointer (return slot) da kullanabilir.
- Fonksiyon gövdesindeki yerel değişkenler fonksiyon dönünce (scope bitince) drop edilir; frame stack'ten kalkar.
- return ile erken çıkışta da aynı drop kuralları işler; o ana kadar oluşturulmuş yerel değişkenler sırayla temizlenir.

Adım 2 — Compiler Davranışı

- Önce soru: "Derleyici neyi kontrol ediyor?" Cevap: her parametrenin ve dönüş değerinin tipi; gövdenin son ifadesinin tipi dönüş tipiyle eşleşmeli.
- ; bir expression'ı statement'a çevirir ve değeri ()'e düşürür. Bu yüzden son satırdaki yanlışlıkla eklenen ; tip hatasına yol açar.
- Dosyadaki fonksiyonlar derleme öncesi bir item (öğe) listesi olarak toplanır; bu yüzden çağrı sırası, tanım sırasından bağımsızdır (ileri referans mümkündür).
- Basit fonksiyonlar genelde inline (gövdeye gömme) edilebilir; derleyici, çağrı maliyetini release build'de optimize edebilir. Bunu garanti eden bir kural yoktur, bir optimizasyondur.
- MIR'de her fonksiyon ayrı bir birim olarak görünür, çağrılar Call terminator'ı ile temsil edilir. Görmek için: cargo rustc -- --emit=mir.

Adım 3 — Hata Kodları

- E0308 (son ifadede ;) : gövde () üretti ama dönüş tipi başka. Çözüm: son ;'yi kaldır veya return kullan.
- E0308 (dönüş tipi yokken değer var): -> Tip eksik ama gövde değer üretiyor. Çözüm: -> Tip ekle veya son ifadeye ; koy.
- E0061: yanlış sayıda argüman. this function takes 2 arguments but 1 argument was supplied. Çözüm: argüman sayısını eşitle.
- E0308 (parametre tipi uyuşmazlığı): add("a", "b") gibi yanlış tipte argüman. Çözüm: doğru tipte argüman ver.
- Kodsuz: missing type for function argument (parametre tipi yazılmadı)
- Detay için: rustc --explain E0308 veya rustc --explain E0061.

Adım 4 — "Neden Böyle?"

- C'de parametre tipi yazmak zorunludur ama dönüş tipi yazmazsan eskiden int varsayılırdı (artık yasak). Rust'ta hem parametre hem dönüş tipi (varsa) her zaman açık yazılır; belirsizlik bırakılmaz.
- C'de return zorunludur; Rust'ta son ifadenin otomatik dönmesi, ternary benzeri kısa ifadeler yazmayı kolaylaştırır (if/match gibi expression'larla birleşir).
- Statement/expression ayrımı, Rust'ın hemen her yapıyı (if, match, block) bir değere çevirebilmesinin temelidir.

- Trade-off: ; unutmak ya da fazladan koymak yeni başlayanlar için sık karşılaşılan bir tuzaktır, ama kuralı öğrenince çok az hataya yol açar.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni Range* tipleri, assert_matches! makrosu, Wasm linker güncellemeleri) fonksiyon tanımlama ve parametre/dönüş tipi kurallarını değiştirmiyor.
- Bu değişiklikler fonksiyonların son-ifade-dönüş kuralını da etkilemiyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: Son satıra yanlılıkla ; koymak

- Neden unutuluyor: her satırı ; ile bitirme alışkanlığı güçlüdür.
- Yol 1 (analoji): ; bir kapıyı kapatır, değeri içeride hapseder; dönüş değeri dışarı çıkamaz.
- Yol 2 (kod): `fn add(a: i32, b: i32) -> i32 { a + b }` (son satırda ; yok)
- Yol 3 (hata): `a + b;` yazarsan → E0308: mismatched types, expected i32, found ()

Sık Unutulan 2: Parametre tipini yazmayı unutmak

- Neden unutuluyor: let içinde tip çoğu zaman yazılmaz, aynı alışkanlık fonksiyona taşınır.
- Yol 1 (görsel): let'te tür çıkarımı var, fn parametresinde yok — fonksiyon imzası bir sözleşmedir.
- Yol 2 (kod): `fn f(x: i32) { }`
- Yol 3 (hata): `fn f(x) { }` → error: expected one of ... found ')

Sık Unutulan 3: Argüman sayısı/tipi uyumsuzluğu

- Neden unutuluyor: Python gibi dillerde varsayılan parametre olabilir, Rust'ta yok.
- Yol 1 (analoji): fonksiyon imzası bir form gibidir; eksik ya da yanlış tipte alan bırakamazsın.
- Yol 2 (kod): `add(3, 4)` — tam iki i32 argüman.
- Yol 3 (hata): `add(3)` → E0061: this function takes 2 arguments but 1 argument was supplied

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da Functions (fn, params, return) konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: Fonksiyon çağrısında stack frame nasıl kurulur? Parametreler nasıl kopyalanır/taşınır? Dönüş değeri nerede tutulur (register vs return slot)? Yerel değişkenler ne zaman drop edilir?
2. COMPILER DAVRANIŞI: Statement/expression ayrımı nasıl çalışıyor? ; bir ifadeyi nasıl () yapıyor? Fonksiyonların tanım sırası neden önemsiz? MIR'de bir fonksiyon çağrısı (Call terminator) nasıl görünür? (--emit=mir)
3. HATA KODLARI: E0308 (iki farklı durum) ve E0061. Her biri için: ne zaman, neden, nasıl çözülür. `rustc --explain EXXXX` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: Zorunlu return, parametre/dönüş tipi yazımı. Rust'ın son-ifade-dönüş kuralı C'den nasıl farklı?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: fabrika hattı, form doldurma).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. Parametresiz, dönüş değersiz bir greet() fonksiyonu yaz ve main'den çağır.
2. İki i32 parametre alıp çarpımını döndüren multiply(a, b) fonksiyonu yaz.



Orta

3. is_even(n: i32) -> bool yaz (n % 2 == 0 son ifade olarak dönsün).
4. max(a: i32, b: i32) -> i32 yaz; if ile return kullanarak erken çıkış yap.



Zor

5. `fn square(x: i32) -> i32 { x * x; }` yaz (bilerek sona ; koy), E0308 hatasını gör ve düzelt. Sonra `main`'den önce tanımlanmış bir `helper()` fonksiyonunu `main` içinden çağır (tanım sırasının önemsizliğini göster). Son olarak 3 parametrelili bir fonksiyonu 2 argümanla çağırıp E0061'i gör.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. Parametre alan, değer döndüren bir fonksiyon yaz.
2. Son ifadeyle dönüş ile `return` ile dönüşü birer örnekte göster.
3. ; fazlalığının neden hata verdiğini söyle.



KAYNAKÇA

Resmi Kaynaklar

- Rust Book — Functions
- Rust Reference — Functions
- Rust Reference — Statements and Expressions

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0308
- E0061

Ek Kaynaklar

- Rust By Example — Functions
- Rust By Example — Statements and Expressions

Not: Bu konu için bu oturumda canlı arama yapılmadı; içerik, Rust Book'un Functions bölümündeki yerleşik kurallara (parametre tipi zorunluluğu, son-ifade-dönüş, tanım sırasının önemsizliği) dayanıyor. Hata mesajlarını (E0308, E0061) ve MIR çıktısını kendi makinende çalıştırarak doğrulaman önerilir.

KONU 11: Ownership (move, Copy, Clone)

Kategori: Bellek | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```

fn main() {
    let s1 = String::from("hello");
    let s2 = s1;                                // move: s1 artık geçersiz
    // println!("{s1}");                        // bu satır açılırsa E0382

    let s3 = s2.clone();                        // clone: gerçek kopya, ikisi de geçerli
    println!("{s2} {s3}");

    let x = 5;
    let y = x;                                // i32 Copy'dir, ikisi de geçerli
    println!("{x} {y}");

    takes_ownership(s3);                      // s3 fonksiyona taşınır
    // println!("{s3}");                        // burada açılırsa E0382

    makes_copy(x);                            // x kopyalanır, burada hâlâ geçerli
    println!("{x}");
}

fn takes_ownership(s: String) {
    println!("got: {s}");
}                                           // s burada drop edilir

fn makes_copy(n: i32) {
    println!("got: {n}");
}

// Çıktı:
// hello hello
// 5 5
// got: hello
// got: 5
// 5

```

Ownership (sahiplik), Rust'ın garbage collector olmadan bellek güvenliği sağlamasını mümkün kılan temel kuraldır: her değerın tam olarak bir owner'ı (sahibi) vardır, owner scope dışına çıkınca değer otomatik drop edilir (serbest bırakılır). `let s2 = s1`; yazınca `String` gibi `Copy` olmayan bir tip `move` (taşınır); artık sadece `s2` geçerlidir, `s1`'i kullanmak derleme hatasıdır. `i32` gibi basit tipler `Copy trait`'ini (özellikliğini) implement ettiği için taşınmaz, bit bit kopyalanır ve ikisi de kullanılabilir kalır. Gerçek bir bağımsız kopya istiyorsan `.clone()` çağırırsın.

1.96.0 notu: Ownership, move ve Copy/Clone kuralları 1.96.0'da aynı.

KATMAN 2 — RECALL KARTLARI

- Move (varsayılan davranış, `Copy` olmayan tipler):

```
let s1 = String::from("hi");
let s2 = s1;
println!("{s1}");
```

Hata: E0382: borrow of moved value: s1 — note: move occurs because s1 has type String, which does not implement the Copy trait

İlgili: .clone(), referans (Konu 12)

- Copy (basit, stack'te yaşayan tipler): i8..i128, u8..u128, f32, f64, bool, char, bunların tuple'ları

```
let x = 5;
let y = x;           // kopya, x hâlâ geçerli
```

İlgili: Copy trait'i Drop trait'i olan tiplerde implement edilemez

- Clone: gerçek, bağımsız kopya oluşturur

```
let s1 = String::from("hi");
let s2 = s1.clone(); // ikisi de geçerli, s1 ayrı bellek
```

Hata: Clone yoksa → E0599: no method named clone found

- Fonksiyona geçirme move sayılır:

```
fn takes(s: String) {}
let s = String::from("x");
takes(s);
println!("{s}"); // E0382
```

İlgili: &s ile borrow et (Konu 12), taşıma olmaz

- Fonksiyondan dönen değer ownership'i geri verir:

```
fn gives() -> String { String::from("mine") }
let s = gives(); // s artık sahip
```

- Kendi struct'ını Copy yapmak:

```
#[derive(Debug, Clone, Copy)]
struct Point { x: i32, y: i32 }
```

Hata: içinde String gibi Copy olmayan alan varsa → E0204: the trait Copy may not be implemented for this type

- Rc<T> paylaşılan sahiplik sağlar (ileri konu, şimdilik sadece adını bil):

```
use std::rc::Rc;
let a = Rc::new(5);
let b = a.clone(); // veri kopyalanmaz, sayaç artar
```

- drop(x); ile elle erken serbest bırakma:

```
let s = String::from("x");
drop(s);
// println!("{s}");    // E0382
```

GoT bağlantı haritası:

ownership  her değerin 1 owner'ı var  scope bitince drop

 move (varsayılan)  Copy olmayan tipler  E0382

 Copy  basit tipler, stack  kopyalanır, taşınmaz

 Clone  .clone()  bağımsız gerçek kopya

fonksiyona geçirme/dönüş  ownership transferi

referans (&)  Konu 12: Borrowing (taşımadan kullan)



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "let s2 = s1; anında bellekte ne oluyor?" Cevap: String'in stack'teki (ptr, len, cap) üçlüsü s2'ye bit bit kopyalanır; heap'teki veri ("hello" metni) hiç taşınmaz, kopyalanmaz. Sadece sahiplik s2'ye geçer ve s1 compiler tarafından "artık geçersiz" olarak işaretlenir.
- Bu yüzden move O(1)'dir: heap allocation (bellek ayırma) yapılmaz. .clone() ise heap'te yeni bir buffer ayırır ve veriyi byte byte kopyalar; bu yüzden pahalıdır.
- i32 gibi Copy tipler tamamen stack'te yaşar; "move" kavramı burada anlamsızdır çünkü kopyalamanın maliyeti taşımayla aynıdır (birkaç byte).
- Scope sonunda owner drop edilince, String'in Drop implementasyonu çalışır ve heap buffer serbest bırakılır (dealloc). Taşınmış bir değer için bu çalışmaz — derleyici "bu zaten taşındı, drop etme" diye işaretler (bu konsept MIR'de drop flag olarak bilinir).

Adım 2 — Compiler Davranışı

- Önce soru: "Derleyici taşınan değeri nasıl takip ediyor?" Cevap: borrow checker her değişkenin "geçerli mi, taşınmış mı" durumunu static olarak (çalıştırmadan) izler; s1 taşındıktan sonraki her kullanım derleme hatasıdır.
- Copy trait'i, derleyiciye "bu tipte let y = x; bir move değil, implicit kopyadır" der. Copy sadece Rust'ın bildiği basit (stack-only) tipler ve bunları içeren struct/tuple'lar için derive edilebilir; Drop implement eden bir tip asla Copy olamaz (ikisi birbirini dışlar).

- `.clone()` derin bir kopya garanti etmez; Clone implementasyonu tipin kendisine bağlıdır (örneğin `Rc::clone` veriyi kopyalamaz, referans sayacını artırır).
- MIR'de taşınan değişkenler için derleyici drop flag'leri ekler; bu da koşullu drop mantığını (hangi dalda taşındı, hangisinde taşınmadı) yönetir. Görmek için: `cargo rustc -- --emit=mir`.

Adım 3 — Hata Kodları

- E0382: taşınmış (veya drop edilmiş) bir değeri kullandın. Çözüm: `.clone()` çağır, referans kullan (Konu 12) veya değeri tekrar atama.
- E0599 (clone yok): tipte Clone implement edilmemiş. Çözüm: `#[derive(Clone)]` ekle (tipi sen tanımlıyorsan).
- E0204: Copy implement edilemeyen bir tipe `#[derive(Copy)]` yazdın (içinde String, Vec gibi heap sahibi bir alan var). Çözüm: Copy'yi kaldır, Clone yeterli.
- E0507 (ileride detaylanacak): borrow edilmiş içerikten değer taşımaya çalışmak (for `x in &v { let y = *x; }` gibi, `x` Copy değilse).
- Detay için: `rustc --explain E0382`.

Adım 4 — "Neden Böyle?"

- C/C++'ta bir pointer'ı kopyalarsan iki pointer aynı belleği gösterir; biri `free()` çağırınca diğeri dangling (sarkık) pointer olur — double-free ve use-after-free hatalarının klasik kaynağı. Rust ownership kuralı bunu derleme zamanında imkânsız kılar: bir değer her zaman tek bir sorumlusu vardır.
- C'in move semantics'i (C11, `std::move`) benzer bir fikri opsiyonel olarak sunar; Rust'ta move varsayılan davranıştır ve derleyici tarafından zorunlu kılınır.
- Garbage collector'lı dillerde (Java, Python) bu sorun runtime'da bir GC ile çözülür; Rust bunu ek bir runtime maliyeti olmadan, compile time'da çözer.
- Trade-off: öğrenme eğrisi diktir (E0382 yeni başlayanların en sık gördüğü hatadır), ama karşılığında çalışma zamanı bellek hataları (use-after-free, double-free, dangling pointer) sınıf olarak ortadan kalkar.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni `Range*` tipleri, `assert_matches!` makrosu, Wasm linker güncellemeleri) ownership ve move kurallarını değiştirmiyor.
- Yeni `Range*` tipleri bazı `core::ops` API'lerini stabilize ediyor; bu, Rust'ın ownership, move ve Copy/Clone kurallarını değiştirmiyor. `assert_matches!` ve Wasm linker güncellemeleri de bu kuralları etkilemiyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: `let s2 = s1;` taşır, kopyalamaz

- Neden unutuluyor: başka dillerde (Python, JS) atama genelde ya referans paylaşır ya da örtük kopyalar, hiçbiri "eskisini geçersiz kılmaz".
- Yol 1 (analoji): s1 bir ev anahtarı; s2 = s1 anahtarı devretmektir, artık s1 elinde anahtar yok.
- Yol 2 (kod): let s2 = s1.clone(); yazarsan ikisi de geçerli kalır.
- Yol 3 (hata): E0382: borrow of moved value: s1

Sık Unutulan 2: Fonksiyona geçirmek de move'dur

- Neden unutuluyor: "sadece değişkenler arası atama taşır" sanılır.
- Yol 1 (görsel): fonksiyon parametresi de bir let gibidir; f(s) aslında let param = s; demektir.
- Yol 2 (kod): takes_ownership(s3); sonrası s3 kullanılamaz; &s3 ile ver, taşınmasın.
- Yol 3 (hata): E0382: use of moved value: s3

Sık Unutulan 3: Hangi tipler Copy, hangileri değil

- Neden unutuluyor: "hepsi basit görünüyor" diye düşünülür.
- Yol 1 (analoji): sabit boyutlu, heap'e dokunmayan değerler (sayılar, bool, char) kartvizit gibidir — kopyalaması bedava. Heap'te veri taşıyan String/Vec birer dosya dolabıdır — kopyalamak pahalı, bu yüzden varsayılan davranış taşımaktır.
- Yol 2 (kod): i32, bool, char, (i32, i32) → Copy; String, Vec<T>, Box<T> → değil.
- Yol 3 (hata): #[derive(Copy)] bir String alanı olan struct'a eklenirse → E0204

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da Ownership (move, Copy, Clone) konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: `let s2 = s1;` anında stack ve heap'te tam olarak ne oluyor? Move neden $O(1)$, clone neden $O(n)$? Drop flag kavramı nedir? Copy tipler için "kopya" ile "move" arasında runtime'da fark var mı?
2. COMPILER DAVRANIŞI: Borrow checker taşınmış değişkenleri nasıl takip ediyor? Copy trait derleyiciye ne söylüyor? Copy ile Drop neden birlikte olamıyor? MIR'de drop flag'ler nasıl görünür? (`--emit=mir`)
3. HATA KODLARI: E0382, E0599, E0204. Her biri için: ne zaman, neden, nasıl çözülür. ``rustc --explain E0382`` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: Pointer kopyalama, double-free, use-after-free, `std::move`. Rust'ın zorunlu move semantics'i bu sorunları nasıl önlüyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: ev anahtarı, fotokopi).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. Bir String oluştur, başka bir değişkene ata (move), eski değişkeni yazdırmayı dene ve E0382'yi gör; sonra kaldırıp düzelt.
2. İki i32 değişken arasında atama yap ve ikisini de yazdır (Copy davranışını göster).



Orta

3. Bir String'i `.clone()` ile kopyala, ikisini de yazdır, sonra birini değiştir (`push_str`) ve diğerinin etkilenmediğini göster.
4. String alan bir fonksiyon yaz, bir String'i ona geçir, sonra orijinali kullanmaya çalışıp E0382'yi gör.



Zor

5. `struct Point { x: i32, y: i32 }` tanımla, `#[derive(Debug, Clone, Copy)]` ekle; iki `Point` arasında atama yap, ikisini de kullan. Sonra `String` alanlı bir `struct`'a bilerek `#[derive(Copy)]` eklemeyi dene, E0204'ü gör ve kaldır. Son olarak `drop(x);` ile elle bir değeri erken serbest bırak, sonra kullanmaya çalışıp hatayı gör.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. `Move`, `Copy` ve `Clone`'u birer kod örneğiyle ayırt et.
2. E0382'nin nedenini açıkla.
3. Hangi tiplerin `Copy` olduğunu listele.



KAYNAKÇA

Resmi Kaynaklar

- Rust Book — What is Ownership?
- Rust Reference — Copy
- `std::clone::Clone`

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0382
- E0599
- E0204

Ek Kaynaklar

- Microsoft Learn — What is ownership?
- itnext.io — Rust Ownership: 50 Code Examples

Not: Move semantics, E0382 mesajı, Copy/Clone farkı ve E0204 davranışı aramada Rust Book ve resmi hata kodu sayfası üzerinden doğrulandı. Heap allocation maliyeti, drop flag kavramı ve MIR görünümü bilgime dayanıyor; size_of ve panic/hata mesajlarını kendi makinende çalıştırarak kontrol et.

KONU 12: Borrowing (& ve &mut)

Kategori: Bellek | Sürüm: Rust 1.96.0

KATMAN 1 — ILK OKUMA

```
fn main() {
    let s1 = String::from("hello");
    let len = calculate_length(&s1);           // &s1: borrow, taşımaz
    println!("'{s1}' has length {len}");       // s1 hâlâ geçerli

    let mut s2 = String::from("hello");
    change(&mut s2);                           // &mut s2: değiştirmeye izinli borrow
    println!("{s2}");

    let r1 = &s1;
    let r2 = &s1;                               // birden fazla immutable referans OK
    println!("{r1} {r2}");
}

fn calculate_length(s: &String) -> usize {    // s: referans, sahip değil
    s.len()
}                                              // s burada drop EDİLMEZ (ödünç aldı)

fn change(s: &mut String) {
    s.push_str(", world");
}

// Çıktı:
// 'hello' has length 5
// hello, world
// hello hello
```

Reference (referans), bir değere sahip olmadan ona işaret eden bir pointer'dır; & ile oluşturulur ve bu işleme borrowing (ödünç alma) denir. `calculate_length(&s1)` çağırısı `s1`'in sahipliğini taşımaz, fonksiyon bitince `s1` hâlâ `main`'de geçerlidir. Varsayılan referanslar `immutable`'dir (değiştirilemez); değiştirmek istiyorsan hem değişkenin hem referansın `mut` olması gerekir, bunu `&mut` ile yaparsın. Aynı anda birden fazla `immutable` referans sorun değildir, ama bir `&mut` referans varken başka hiçbir referans (ne `&` ne `&mut`) olamaz — bu kural Konu 13'te detaylandırılacak.

1.96.0 notu: Borrowing kuralları 1.96.0'da aynı.

KATMAN 2 — RECALL KARTLARI

- Immutable referans: `&T` — okur, değiştiremez

```
fn f(s: &String) { println!("{s}"); }
```

- Mutable referans: `&mut T` — hem değişken hem referans `mut` olmalı

```
let mut s = String::from("hi");
let r = &mut s;
r.push_str("!");
```

Hata: `s` `mut` değilse → E0596: cannot borrow as mutable, as it is behind a `&` reference / not declared as mutable

- Aynı anda tek bir `&mut`:

```
let mut s = String::from("hi");
let r1 = &mut s;
let r2 = &mut s; // ikinci mutable referans
println!("{r1} {r2}");
```

Hata: E0499: cannot borrow `s` as mutable more than once at a time

- `&mut` varken `&` olamaz (ve tersi):

```
let mut s = String::from("hi");
let r1 = &s;
let r2 = &mut s; // immutable varken mutable istedi
println!("{r1}");
```

Hata: E0502: cannot borrow `s` as mutable because it is also borrowed as immutable

- NLL (Non-Lexical Lifetimes): referansın ömrü son kullanıldığı yerde biter, süslü parantezin sonunda değil:

```
let mut s = String::from("hi");
let r1 = &s;
println!("{r1}");           // r1'in son kullanımı burada

let r2 = &mut s;           // OK: r1 artık "ölü"
println!("{r2}");
```

- Dereference (*) ile değere ulaşmak:

```
let x = 5;
let r = &x;
println!("{}", *r);        // 5; println! otomatik deref de yapar: {r} de olur
```

- Fonksiyondan referans dönmek (dangling referans yasak):

```
fn dangle() -> &String {      // E0106: missing lifetime specifier
    let s = String::from("x");
    &s
}                             // s burada drop edilir, referans sarkık kalırdı
```

Çözüm: String'i doğrudan döndür (ownership transfer), referans değil

- Metot çağrılarında otomatik borrow:

```
let s = String::from("hi");
s.len();                // aslında (&s).len() gibi çalışır, taşmaz
```

GoT bağlantı haritası:

borrowing  &T (immutable)  birden fazla OK
 &mut T (mutable)  tek seferde sadece 1 tane (E0499)
 & ile aynı anda olamaz (E0502)

reference  sahip değil  scope bitince orijinali drop etmez

NLL  referansın ömrü son kullanımda biter

dangling referans yasak  E0106 (lifetime, konu 25'te detaylı)



KATMAN 3 — DERİN TEKNİK NOT

Adım 1 — Bellek Seviyesi

- Önce soru: "Referans bellekte ne?" Cevap: bir referans, işaret ettiği değerin bellek adresini tutan bir pointer'dır. `&i32` 8 byte (64-bit adres); `&str` gibi fat pointer'lar 16 byte (adres + uzunluk).
- `&s1` oluşturmak hiçbir veri kopyalamaz, hiçbir heap allocation yapmaz; sadece `s1`'in adresini taşıyan küçük bir değer üretir.
- Fonksiyona `&String` geçirmek, çağrılan fonksiyonun stack frame'ine sadece bu adresi kopyalar; `String`'in kendisi (heap'teki veri) yerinde kalır ve taşınmaz.
- `&mut` referans da aynı boyutta bir pointer'dır; fark bellekte değil, derleyicinin bu pointer üzerinden yazmaya izin vermesindedir.

Adım 2 — Compiler Davranışı

- Önce soru: "Borrow checker ne kontrol ediyor?" Cevap: her değer için, herhangi bir anda ya (a) sınırsız sayıda `&` referans ya da (b) tam olarak bir `&mut` referans olabileceğini, ikisinin asla aynı anda olamayacağını doğrular.
- Bu, data race'leri (veri yarışı) compile time'da önler: aynı veriye aynı anda hem okuma hem yazma erişimi (ya da iki yazma erişimi) asla oluşamaz.
- NLL sayesinde borrow checker, referansın söz dizimsel scope'una değil gerçek son kullanım noktasına bakar; bu, eskiden hata veren birçok geçerli kodu artık kabul eder.
- Fonksiyondan `&` dönmek, lifetime annotation (yaşam süresi işaretleyici) gerektirir (Konu 25); işaretlenmezse ve derleyici çıkaramazsa E0106 verir.
- MIR'de referanslar Ref ve `&mut` referanslar Mut Ref olarak, borrow checker'ın kendisi ayrı bir analiz aşaması (borrowck) olarak çalışır. Görmek için: `cargo rustc -- --emit=mir` (borrow checker hatalarını görmek için zaten normal derleme yeterli).

Adım 3 — Hata Kodları

- E0596: mut olmayan bir değişkene/referansa mutable borrow almaya çalıştın. Çözüm: değişkeni let mut yap.
- E0499: aynı değere aynı anda iki `&mut`. Çözüm: ilkini son kullandıktan sonra ikincisini al (NLL buna izin verir) veya scope'ları ayır.
- E0502: `&` ve `&mut` aynı anda çakişti. Çözüm: immutable referansların son kullanımını mutable referanstan önceye al.
- E0106: fonksiyon dönüş tipinde referans var ama lifetime belirtilmemiş. Çözüm: ownership döndür veya lifetime annotation ekle (Konu 25).
- Detay için: `rustc --explain E0502`.

Adım 4 — "Neden Böyle?"

- C/C++'ta bir pointer'ı kaç yere istersen kopyalayabilirsin; biri veriyi değiştirirken başka bir thread veya kod yolu aynı anda okursa data race oluşur — bu klasik eşzamanlılık hatalarının kaynağıdır.
- C++'ın `const T&` ve `T&` ayrımı kavramsal olarak benzer, ama derleyici aynı anda bir `T&` ile bir `const T&`'nin aynı veriye bakmasını engellemez; Rust bunu borrow checker ile zorunlu kılar.

- Rust'ın kuralı "aynı anda ya çoklu okuyucu ya tek yazıcı" (reader-writer lock mantığına benzer), ama kilitleme (locking) runtime'da değil, compile time'da gerçekleşir — çalışma zamanı maliyeti sıfırdır.
- Trade-off: bazı geçerli-görünen desenler (örneğin bir struct'ın iki farklı alanına aynı anda mutable erişim) derleyiciyi ikna etmek için yeniden yazılmalıdır, ama karşılığında data race sınıfı hatalar derleme zamanında elenir.

Adım 5 — Rust 1.96.0 Farkı

- Rust 1.96.0'daki değişiklikler (yeni Range* tipleri, assert_matches! makrosu, Wasm linker güncellemeleri) bu konuyu etkilemiyor. Borrowing kuralları, NLL davranışı ve E0499/E0502/E0596/E0106 hata kodları 1.96.0'da değişmedi.
- Rust 1.96.0'daki değişiklikler borrowing kurallarını veya borrow checker davranışını değiştirmiyor.



KATMAN 4 — SIK UNUTULANLAR + DERİNLEŞTİRME

Sık Unutulan 1: Referans almak taşımaz

- Neden unutuluyor: Konu 11'den sonra her şeyin "taşınacağı" hissine kapılınabilir.
- Yol 1 (analoji): &s bir kitabı ödünç vermektir, s taşımak kitabı hediye etmektir. Ödünç veren kitabı geri alır.
- Yol 2 (kod): fn f(s: &String) {} çağrısından sonra orijinal değişken hâlâ kullanılabilir.
- Yol 3 (hata): referans yerine değeri taşırsan (fn f(s: String)), sonraki kullanımda E0382 görürsün (Konu 11).

Sık Unutulan 2: &mut tekil kuralı

- Neden unutuluyor: "sadece okuyorum, sorun olmaz" diye iki &mut birden alınabileceği sanılır.
- Yol 1 (görsel): bir anda sadece bir kişi tahtaya yazabilir; birden fazla kişi aynı anda yazarsa karmaşa çıkar.
- Yol 2 (kod): ilk &mut'ı kullandıktan sonra (NLL ile "öldükten" sonra) ikincisini al.
- Yol 3 (hata): E0499: cannot borrow s as mutable more than once at a time

Sık Unutulan 3: & ve &mut karışması

- Neden unutuluyor: "bir tanesi okuma, diğeri yazma, farklı işler" diye ayrı sorun sanılmaz.
- Yol 1 (analoji): bir kişi deftere bakarken (okurken) başka biri aynı anda üzerine yazarsa, okuyan kişi yanlış/eksik veri görebilir.
- Yol 2 (kod): immutable referansların println!'ini mutable referanstan önce bitir.
- Yol 3 (hata): E0502: cannot borrow s as mutable because it is also borrowed as immutable

Hazır Derinleştirme Prompt'u:

Rust 1.96.0'da Borrowing (& ve &mut) konusunu en detaylı şekilde anlat. Şunları kapsa:

1. BELLEK SEVİYESİ: Bir referans bellekte kaç byte, ne tutar? &T ile &mut T'nin bellek temsili aynı mı? Referans almak neden heap allocation gerektirmiyor?
2. COMPILER DAVRANIŞI: Borrow checker "çoklu & veya tek &mut" kuralını nasıl uyguluyor? NLL (Non-Lexical Lifetimes) referansın ömrünü nasıl belirliyor? Data race'ler compile time'da nasıl önleniyor? MIR'de referanslar ve borrowck nasıl çalışır?
3. HATA KODLARI: E0596, E0499, E0502, E0106. Her biri için: ne zaman, neden, nasıl çözülür. `rustc --explain EXXX` çıktısını da ekle.
4. C/C++ KARŞILAŞTIRMASI: Pointer/referans kopyalama, data race, const T& / T&. Rust'ın borrow checker'ı bu sorunları nasıl compile time'da çözüyor?
5. ANALOJİ: Günlük hayattan bir benzetme yap (örn: kitap ödünç verme, ortak defter).
6. RUST 1.96.0: Bu konuda son sürümde değişen bir şey var mı?
7. PRATİK: 3 tane kod örneği ver — kolay, orta, zor.

Açıklama Türkçe, teknik terimler İngilizce, kod yorumları Türkçe olsun.

Her bölümü ayrı başlıkla ver. Kod bloklarını çalıştırılabilir yaz.



GÖREV (Boş dosyaya yaz)



Kolay

1. Bir String oluştur; uzunluğunu &s alan bir fonksiyonla hesapla ve orijinal değişkeni fonksiyondan sonra da yazdır.
2. Bir mut String oluştur; &mut alan bir fonksiyonla ona push_str ile metin ekle ve sonucu yazdır.



Orta

3. Aynı String'e iki tane & referans al, ikisini de aynı println!'de kullan.
4. let mut s ile başla; önce &s al ve kullan, kullanımı bitince &mut s al ve kullan (NLL'yi göster).



Zor

5. Bilerek aynı anda iki `&mut s` almayı dene ve E0499'u gör; düzelt. Sonra bir `&s` dururken `&mut s` almayı dene ve E0502'yi gör; immutable referansın kullanımını öne alarak düzelt. Son olarak bir fonksiyondan `&String` döndürmeyi dene (dangle örneği) ve E0106'yı gör.

Başarı kriteri: Hepsini hatasız yazarsan → öğrendin ☑

☑ Kendini Test Et (2-3 saat sonra, sıfırdan)

1. `&` ve `&mut` parametrelili birer fonksiyon yaz.
2. "Çoklu `&` veya tek `&mut`" kuralını bir örnekle göster.
3. E0499 ve E0502'nin farkını açıkla.



KAYNAKÇA

Resmi Kaynaklar

- Rust Book — References and Borrowing
- Rust Reference — References

Sürüm Notları

- Rust Blog — Announcing 1.96.0
- Rust 1.96.0 Release Notes

Hata Kodları

- E0596
- E0499
- E0502
- E0106

Ek Kaynaklar

- Rust By Example — Borrowing

Not: "Çoklu immutable veya tek mutable referans" kuralı, E0499, E0502 mesajları ve NLL davranışı (son kullanıma göre ömür) aramada Rust Book üzerinden doğrulandı. Referans boyutları (8/16 byte), E0596/E0106 ve MIR/borrowck detayları bilgime dayanıyor; size_of ve hata mesajlarını kendi makinende çalıştırarak kontrol et.
